The Limits of Predictions for Online Bipartite Matching

A Unified Experimental Study

Zhuolun Li

A dissertation submitted to the University of Bristol in accordance with the requirements for award of the degree of MSc in Computer Science in the Faculty of Science and Engineering.

School of Computer Science

2026-08-28

Word count: 11889

Executive Summary

Online bipartite matching is the canonical model of irrevocable sequential allocation: resources are known in advance, requests arrive one at a time, and each must be matched to a free compatible resource (or dropped) before the rest of the input is seen. It underlies online advertising, ride-hailing dispatch and request routing. A recent line of work hands such an algorithm a *prediction* about the input, asking for near-optimal performance when it is good (*consistency*) and no loss when it is wrong (*robustness*).

We study two such families whose prior analyses used separate settings and focused largely on worst-case theory, leaving their empirical comparison open. This project puts them on one harness (same graphs, arrivals, optimum and paired seeds, four structured error models, 95% confidence intervals), validated against a published study and then run on synthetic families, six real-world graphs and real traces. The same broad pattern recurs across the settings examined: the advice-free baseline is already near-optimal (0.990 of the offline optimum), leaving good advice under 0.01 to add, whereas unguarded prediction-following collapses to 0.472. In these experiments, predictions act primarily as **robustness insurance rather than as a performance lever**.

- I built the harness and validated it against the published study of Borodin, Karavasilis and Pankratov before adding any prediction work (pages 10–16).
- I created a unified benchmark of the two families, implementing the published algorithms of Aamand–Chen–Indyk, Choo et al. and Burathep–Erlebach–Moses on a common error model and optimum; every wide gap it shows is a *downside* gap (pages 17–21).
- I characterised what governs the residual loss: a Kendall-τ order error (Spearman $\rho = 0.979$), with the known $n - \text{LIS}$ bound loose by 16–75×; order-dependence is Aamand–Chen–Indyk's (pages 22–24).
- I conducted an empirical study of test-and-fallback (a pathology in which a *more accurate* test decides *worse*, and a resolution limit; pages 25–30), and established external validity on six real graphs and a cheap non-ML predictor (pages 31–34).
- I report two negative results (pages 35–43), prove a general consistency/robustness trade-off inequality, and give a bounded interpretation—not a further theorem—that predicts a scale of ≈ 0.004, the same order as the measured upsides (pages 45–50).

Acknowledgements

I would like to thank my supervisor, Christian Konrad, for his guidance, thoughtful feedback, and support throughout this project.

Statement on the inclusion of published and collaborative work

My dissertation does not include work that has been published, submitted or prepared for publication with me as an author. My dissertation does not include non-published collaborative work that contains data or contributions from others.

If there are any publications with me as an author and/or any collaborative work that contains data or contributions from others, I have completed the following table.

Chapter	**Details**	**Author Contributions**

The information in this table counts as the references to my own publications if they are used as whole chapters or as whole sections of chapters. If work from my own publications have been incorporated in smaller segments, I have included individual references to them in the relevant part of the dissertation in addition to completing the table.

Author's Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

Zhuolun Li

2026-08-28

Contents

List of Figures

1 Introduction

1.1 Background and motivation

Online bipartite matching is the canonical model of irrevocable sequential allocation. Resources are known in advance; requests arrive one at a time; each must be matched to an available compatible resource (or dropped) immediately, before the rest of the input is seen. It is the abstraction behind online advertising, ride-hailing dispatch, organ exchange, and request routing in modern serving systems. Because decisions cannot be undone, the difficulty lies entirely in committing well under uncertainty about the future.

A recent and influential idea is to give such an algorithm a **prediction** about the input (which resources will be contended, or how many requests of each kind will arrive), and to ask for two guarantees at once: **consistency**, meaning near-optimal performance when the prediction is good, and **robustness**, meaning no worse than a prediction-free algorithm when the prediction is wrong. (We call the object a *prediction* throughout, and *advice* only where the question is whether to trust it; the two denote the same thing.) For online matching specifically, two families of such *learning-augmented* algorithms have appeared: MinPredictedDegree, which consumes a prediction of each resource's degree (how many kinds of request will compete for it, i.e. how contended it will be), and a family of *test-and-fallback* schemes, which test a request-type prediction on a short prefix of arrivals before deciding whether to trust it.

These algorithms come with attractive worst-case guarantees, yet, as this thesis demonstrates experimentally, their average-case behavior is strikingly muted: on realistic workloads the sophisticated predictions rarely *help*, while the algorithms' real value is that they do not *hurt*. This gap between the worst-case promise and the average-case reality is the puzzle that motivates this thesis.

Moreover, the published work we examined studied these algorithms separately: each on its own input model and its own notion of prediction error, and largely through theory. There was therefore no common empirical basis on which to compare them or quantify how much a prediction buys. This thesis provides that basis and investigates the gap.

1.2 Research objectives

Performance is measured throughout as the ratio of the matching an algorithm produces to the offline optimum on the same instance (§3.1 makes this precise). The thesis is organized around three questions:

1. **How do the learning-augmented online-matching algorithms actually compare**, head to head, under a single prediction-error model and on common inputs, and how much does a prediction help, at what cost, on realistic data?
2. **What are the failure modes of the adaptive test-and-fallback mechanism**: the cost of its test, the calibration of its accept/reject decision, and where it goes wrong?
3. **Why does the average-case experience differ so sharply from the worst-case promise**: is the observed wall an artifact of particular algorithms and generators, or is it necessary?

The first two are answered experimentally. The third receives a bounded theoretical treatment—one proved trade-off inequality and a quantitative interpretation—but is not closed.

1.3 Contributions

- **A unified experimental benchmark** (Chapter 4; Q1): a head-to-head comparison of these families, on one experimental harness: a single code path in which every algorithm sees the same graphs, predictions and optimum, with a *structured* error model (errors follow the instance's structure, not i.i.d. noise) and confidence intervals.

- **A characterization of predictions as robustness insurance** (Chapters 4, 7; Q1): unguarded following crashes below the advice-free baseline, two mechanisms (structural and adaptive) restore safety with different trade-offs, and the consistency upside is small, so the value is downside protection. Validated on synthetic graphs, six real-world graphs and real traces, including with a cheap, non-ML historical predictor.

- **An empirical engagement with the order-error theory** (Chapter 5; Q1): MinPredicted-Degree's loss is governed by a Kendall-τ order error, the fraction of resource pairs ranked in the wrong relative order, onto which several error models collapse. Order-dependence itself is Aamand–Chen–Indyk's result; ours is the characterization of their bound's looseness and saturation, and of the right order measure.

- **An empirical study of test-and-fallback** (Chapter 6; Q2): its testing cost, a threshold-calibration pathology in which a *more accurate* test yields a *worse* decision, the resolution limit that recalibration exposes, and an irrevocability penalty that explains why matching requires *test-then-commit*.

- **A quantified account of why the wall stands** (Chapter 6; Q3). By the *wall* we mean that on average-case inputs neither a better algorithm nor a better prediction moves the ratio appreciably. We trace it to a *resolution limit* (no practical threshold can capture an upside below its own estimator's noise) that persists across the whole difficulty range (Figure 6.4).

Alongside these, the thesis documents (Chapter 8) the exploratory directions it pursued and the negative results they returned: an attempt to learn the predictor for extra performance, and an attempt to find a serving regime where predictions genuinely help. Both reinforce the central finding, and the question they raise — is the wall forced? — is taken up in the concluding outlook (§10.2). That outlook is a discussion rather than a further contribution: it proves one consistency/robustness trade-off inequality and then reads our own experiments through it. No theorem beyond that inequality is claimed anywhere in this thesis.

1.4 Thesis organization

The chapters group by the three questions above. Chapters 2 and 3 build the common ground: the background (input models, the learning-augmented paradigm, the specific algorithms, and the distribution-testing results behind their tests) and then our own model, methodology and harness, validated by reproducing a subset of the Borodin et al. study. Chapters 4 to 7 answer Q1 and Q2: the unified benchmark and its four findings (4), what governs the small loss (5), the test-and-fallback mechanism in depth (6), and external validity on real predictors and real graphs (7). Chapters 8 to 10 take up Q3 and the boundaries: the exploratory directions and their negative results (8), the AI-inference serving case study (9), and a conclusion that summarizes, gives a theoretical outlook on why the wall stands (§10.2), evaluates the project critically against these objectives (§10.3), and states limitations and future work (10). One sentence recurs throughout, and §10.1 states it in full: **on average-case online matching, predictions are robustness insurance rather than a performance lever.**

2 Background and Related Work

This chapter sets up the technical context the thesis builds on: the online bipartite-matching problem and its input models (§2.1), the known-i.i.d. model we work in (§2.2), the learning-augmented ("with-predictions") paradigm and its consistency/robustness language (§2.3), the specific prediction-based matching algorithms we benchmark (§2.4), the distribution-testing results behind the algorithms' tests and the outlook of §10.2 (§2.5), and the gaps in the literature this thesis addresses (§2.6).

2.1 Online bipartite matching and competitive analysis

In online bipartite matching, one side of a bipartite graph (the *offline* resources) is known in advance, while the vertices of the other side (the *online* requests) arrive one at a time. On each arrival the algorithm sees the request's incident edges and must either match it to a currently unmatched neighbor or leave it unmatched, **immediately and irrevocably**. The goal is to maximize the size (or, in weighted variants, the value) of the matching. Performance is measured by the **competitive ratio**: the ratio between the algorithm's matching and an offline optimum on the same input, in expectation over both the random input and the algorithm's own randomness (§3.1 fixes the precise form we report).

The difficulty of the problem depends entirely on the assumed *input model*:

- **Adversarial arrival order.** The requests and their order are chosen by an adversary. The seminal result of Karp, Vazirani and Vazirani [12] is that the randomized Ranking algorithm (fix a uniformly random priority order on the resources and match each request to its highest-priority available neighbor) achieves competitive ratio $1 - 1/e \approx 0.632$, and that this is optimal for the adversarial model. Deterministic algorithms cannot beat $1/2$.

- **Random arrival order.** The graph is adversarial but the requests arrive in a uniformly random order; this is strictly easier than adversarial order and admits ratios above $1 - 1/e$.
- **Known-i.i.d.** Requests are drawn independently from a known distribution over request types (§2.2); this is the average-case model and the one this thesis studies.

These three models are nested in difficulty, and the direction matters throughout the thesis: every known-i.i.d. instance is also a random-order instance, and every random-order instance is an adversarial-order instance. A guarantee proved in a harder model therefore holds in an easier one, but not conversely, so a bound proved for adversarial order says nothing about how well an algorithm does on known-i.i.d. inputs beyond that floor.

2.2 The known-i.i.d. model

In the known-i.i.d. model there is a bipartite *type graph*: each online **type** ℓ has a fixed neighborhood $N(\ell)$ among the offline resources, and an instance is generated by drawing m arrivals independently from a known distribution p over types. The type graph and p are known; the realized sequence is not. This models settings (ad serving, recurring request streams) where the population of requests is statistically stable even though individual arrivals are not.

A line of work has pushed the worst-case (over type graphs) competitive ratio above the $1 - 1/e$ adversarial barrier: Feldman et al. [8] first beat it (0.670) via a flow-based *blue/red* decomposition of a suggested matching; Manshadi, Oveis Gharan and Saberi [14] and Jaillet–Lu [10] improved the ratio (to ≈ 0.702 and ≈ 0.729 respectively) using LP-based and list-based online policies; subsequent work refined it further. These are the "sophisticated" algorithms whose worst-case optimality motivates their use.

Crucially for this thesis, Borodin, Karavasilis and Pankratov [2] conducted an experimental study of these algorithms and found that on average-case i.i.d. instances the picture is very different from the worst case: the simple algorithms (Greedy, Ranking) perform almost as well as the sophisticated ones, whose advantage is a worst-case, not a typical-case, phenomenon.

One empirical observation in that line is the seed of this thesis: *on typical inputs the simple baseline is already near-optimal.* Everything that follows is an attempt to find out what a prediction can add to it, and we reproduce a subset of the study as validation (Chapter 3).

2.3 Learning-augmented algorithms

The **algorithms-with-predictions** (or learning-augmented) paradigm, surveyed in [16], equips an online algorithm with a prediction about the unknown input and asks for two guarantees simultaneously:

- **Consistency:** near-optimal performance when the prediction is accurate;
- **Robustness:** performance no worse than a prediction-free guarantee when the prediction is arbitrarily wrong.

The paradigm was crystallized by Lykouris and Vassilvitskii [13] for competitive caching, who showed how to interpolate between trusting and ignoring a predictor while bounding both consistency and robustness. A large literature followed across ski rental, scheduling and other online problems, including the optimal consistency/robustness trade-off analyses of Wei and Zhang [22], which establish problem-intrinsic Pareto frontiers between the two objectives. Recent work further analyzes how the achievable ratio degrades continuously with prediction accuracy in a distributionally-robust formulation [23], complementing the discrete consistency/robustness endpoints used here.

Two items from this literature are load-bearing for us. The recurring mechanism is *hedging*: combining the prediction with a safe default so that a bad prediction cannot cause catastrophe. The blind-follow-with-switching combiner of Chłędowski, Polak, Szabucki and Żołna [5] is one such mechanism, which we port and benchmark (Chapter 6); their paper, an experimental study of robust learning-augmented caching, is also the closest methodological template for this thesis's experimental half.

2.4 Predictions for online bipartite matching

Two strands apply the with-predictions paradigm to online matching, differing in the *prediction object* they consume.

Degree predictions: MinPredictedDegree. Aamand, Chen and Indyk [1] propose **MinPredictedDegree (MPD)**: given a prediction μ of each offline resource's degree (how contended it will be), match each arrival to its available neighbor of *minimum predicted degree*, i.e. protect the resources predicted to be rarest. MPD is robust by construction: a constant (useless) predictor reduces it to Ranking. A key structural fact, which the thesis engages in Chapter 5, is that MPD depends on μ *only through the order it induces*. The authors' Appendix D bounds the matching loss by an order quantity built in two steps: list the true degrees in the order the prediction suggests, then count how many of them are out of place: formally $n - \mathrm{LIS}$, where LIS is the length of the longest non-decreasing subsequence of that list. The count is zero exactly when the prediction gets the order right, so a monotone (order-preserving) prediction incurs zero loss.

Type-histogram advice and test-and-fallback. A second strand takes the prediction to be a histogram $\hat{c}$ over request types (how many of each type will arrive). Choo et al. [6] introduce TestAndMatch: build a matching from $\hat{c}$ and Mimic it, but first spend a sublinear prefix of arrivals testing whether the observed type frequencies match $\hat{c}$, using an ℓ_1-distance tester adapted from Jiao, Han and Weissman [11], and fall back to Ranking if they do not. Burathep, Erlebach and Moses [3] generalize this ("Test-and-Match+") to the random-arrival model and to imperfect knowledge of the matching size. Both papers give *upper* bounds (algorithms); their only lower bound (Choo et al.'s Theorem 3.1) is a generic adversarial indistinguishability result (no algorithm is 1-consistent and $> \frac{1}{2}$-robust), and neither proves a lower bound in the stochastic model. Notably, Choo et al.'s acceptance threshold already couples to the baseline competitive ratio β, but constructively, inside the algorithm design, not as a lower bound. What that coupling costs, i.e. how large a prefix the follow/fallback decision fundamentally requires, is the question this thesis measures (Chapter 6) and then reads quantitatively (§10.2).

2.5 Distribution testing

The test at the heart of the algorithms above is a question in **distribution testing**: given samples from an unknown distribution p over a support of size r (the same r as in our model, the number of request types, because the histogram advice is a distribution over types) and a known reference q, decide how far p is from q. We measure that distance in ℓ_1 throughout, as the algorithms and our experiments do; it is *twice* the total-variation distance, and every threshold quoted in this thesis is an ℓ_1 threshold. Two regimes must be distinguished:

- **Identity (non-tolerant) testing**, distinguishing $p = q$ from $\|p - q\|_1 \geq \varepsilon$, has sample complexity $\Theta(\sqrt{r}/\varepsilon^2)$ [17, 21], *sublinear* in the support size.

- **Tolerant testing / distance estimation**, distinguishing $\|p-q\|_1 \leq \varepsilon_1$ from $\geq \varepsilon_2$ for $0 < \varepsilon_1 < \varepsilon_2$, i.e. estimate the distance rather than test equality, is provably much harder: Valiant and Valiant [20] showed it requires $\Theta(r/\log r)$ samples, *near-linear* in the support. Jiao, Han and Weissman [11] gave matching bounds for ℓ_1-distance estimation, and Canonne, Jain, Kamath and Li [4] precisely characterized the whole $(\varepsilon_1, \varepsilon_2)$ landscape, showing that for constant tolerances the cost jumps to the "barely sublinear" $\tilde{\Theta}(r/\log r)$.

The near-quadratic gap between $\sqrt{r}$ (testing) and $r/\log r$ (tolerant testing) matters twice in this thesis. It is why the deployable versions of TestAndMatch fall back to an empirical surrogate for their tester, and it is why that surrogate is blind on large supports: the resolution limit measured in Chapter 6. Whether the barrier also dooms every other decision rule turns out to be subtler than it first appears; the concluding outlook (§10.2) returns to this point.

2.6 Positioning of this thesis

Against this background, three gaps stand out. They correspond one-to-one to the three questions of §1.2, and the thesis addresses them in that order:

1. **No unified empirical comparison found in the reviewed literature.** The matching algorithms above were studied on their own input families and error models, and largely in theory; we did not identify a head-to-head benchmark under a common harness. (Chapters 4–7.)
2. **No empirical study of test-and-fallback found in the reviewed literature.** The distribution test at the heart of [6, 3] has no deployable implementation (the authors themselves fall back to an empirical surrogate), and its testing cost, threshold calibration, and failure modes have not been measured. (Chapter 6.)
3. **No quantification of what the prefix test costs.** No prior work measures, or bounds, how large a prefix the follow/fallback decision requires on strong-baseline instances, where the upside to be captured is smallest (§8.3 describes the prior-art pass behind this claim). (Chapter 6 empirically; §10.2 in outlook.)

3 Model, Algorithms, and Methodology

Chapter 2 placed the problem in its field. This chapter fixes the precise model and notation used throughout (§3.1), details the algorithms as we implement them (§3.2), the prediction objects and structured error models (§3.3), the consistency/robustness measures (§3.4), and the experimental harness (§3.5). It closes by validating the harness against the published Borodin et al. figures before any contribution is built on it (§3.6).

3.1 The known-i.i.d. model, precisely

There is a set R of n offline **resources** and a **type graph** on r online **types**; type ℓ has neighborhood $N(\ell) \subseteq R$. An instance is generated by drawing m arrivals i.i.d. from a distribution p over the r types; the i-th arrival reveals its type (hence $N(\cdot)$) and must be matched to a currently unmatched resource in its neighborhood, immediately and irrevocably, or left unmatched. The type graph and p are known to the algorithm; the realized arrival sequence is not. Following the standard *integral-types* convention, the default is $m = n$ with uniform p, so each type's expected count is an integer; the classical setting has $r = n$ (each offline vertex a distinct type), and the *few-types* setting used for the histogram-advice experiments has $r \ll n$.

The performance measure is the **competitive ratio**, computed per instance and then averaged: $\rho(\mathrm{ALG}) = \mathbb{E}[|\mathrm{ALG}|/|\mathrm{OPT}|]$, where OPT is a maximum matching of the realized instance, computed exactly by Hopcroft–Karp [9]. Averaging per-instance ratios is the natural convention under paired trials (§3.5), since on a given instance every algorithm is divided by the same exactly-computed optimum. It differs in principle from the ratio of expectations $\mathbb{E}[|\mathrm{ALG}|]/\mathbb{E}[|\mathrm{OPT}|]$ that an experimental study might report instead, so we measured the gap: across nine algorithm-by-quality cells spanning both prediction families, the two conventions agree to within 6×10^{-5}, two orders of magnitude below the confidence intervals we report, so no number in this thesis depends on the choice (`scripts/run_metric_check.py`, §A.2). The advice-free baseline throughout is KVV Ranking, whose ratio we write ρ_{base} (the **baseline strength**).

The notation recurs throughout the thesis and is collected here for reference:

symbol	meaning	typical value
n	offline resources	1000–2000
r	online request types	$r = n$ (classical), $r = 8$ (few-types)
m	arrivals in one instance	$m = n$
p	known distribution over types	uniform unless stated
μ	predicted resource degrees (degree prediction)	—
$\hat{c}$	predicted type counts (histogram prediction)	—
k	length of the prefix a test may inspect	$k = 200$
η	strength of the injected prediction error	$\eta \in [0, 1]$
ρ_{base}	Ranking's ratio on that instance family (the floor)	0.89–0.99

3.2 Algorithms

All greedy-type algorithms are instances of one primitive: given a total order (**rank**) on the resources, GreedyWithPermutation matches each arrival to its available neighbor of minimum rank. This makes the algorithm family a single code path parameterized by the rank, which is important both for a fair comparison and for the theory.

- **Advice-free baselines.** SimpleGreedy (identity rank), Ranking (uniformly random rank;

the baseline), and the stochastic-matching algorithms Feldman and Jaillet–Lu, which precompute a suggested matching by max-flow: Feldman from a cap-$\{2,1,2\}$ network, whose unit-flow subgraph splits into a blue and a red matching; Jaillet–Lu from a cap-$\{3,2,3\}$ network, whose fractional solution gives a per-type list of resources to try in order. Each of the two has a *non-greedy* variant (follow the suggestion only) and a *greedy* variant (fall back to any available neighbor).

- **Degree predictions.** MinPredictedDegree (MPD) is GreedyWithPermutation ranked by ascending predicted degree $\mu \in \mathbb{R}^R$. Constant μ gives Ranking; the true realized degrees give the consistency ceiling (MinDegree). The **augmentations** Feldman(MPD) and JailletLu(MPD) use the MPD rank only as the tie-break of the greedy fallback.
- **Type-histogram advice and test-and-fallback.** From a count vector $\hat{c}$ over types we build an advice matching $\hat{M}$ and record its per-type partners. FollowPrediction routes every arrival to its type's predicted partner; TestAndMatch (Choo and BEM) does the same over a length-k prefix, tests the empirical ℓ_1 distance between the prefix frequencies and $q = \hat{c}/\|\hat{c}\|_1$ against a threshold τ, then continues or falls back to Ranking. The Chłędowski-style dynamic combiner is ported as a benchmarked baseline, not a contribution.

This last point is the mechanism behind the structural robustness of Chapter 4. Because the prediction only decides between otherwise equivalent choices, the worst-case-optimal base matching carries most of the load and a bad prediction can affect only the tie-breaks. This limits the downside, but also caps the upside when the prediction is good.

3.3 Prediction objects and structured error models

The families consume different prediction objects (degree vectors μ and type histograms $\hat{c}$), so each is corrupted by its own knob and reported in a parallel panel. For μ we use four structured error models, each driven by a strength $\eta \in [0,1]$ and each returning both an ℓ_1 *magnitude* error and a normalized **Kendall-τ order error**:

model	construction at strength η	effect on the induced order
random-flip	independently, with probability η, replace an entry of μ by a uniform draw from $[\min \mu, \max \mu]$	grows with η; $\eta = 1$ is a constant-quality predictor, i.e. Ranking
systematic-bias	monotone rescale $\mu \leftarrow \mu \cdot (1 + \eta)$	none: Kendall-$\tau \equiv 0$ by construction, while the ℓ_1 magnitude grows linearly
adversarial	for an η-fraction of resources $u \in R$, reflect $\mu(u) \leftarrow (\min \mu + \max \mu) - \mu(u)$	the most order-damaging perturbation at a given fraction; $\eta = 1$ reverses the order
distribution-drift	blend toward the degrees of an independently drawn graph of the same family, $\mu \leftarrow (1 - \eta)\mu + \eta\mu_{\text{alt}}$	grows with η, but in a correlated way rather than entry-by-entry

For $\hat{c}$ we blend the realized counts toward a concentrated random target by a fraction $\eta \in [0, 1]$, reporting the induced $\ell_1(p, q)$. Because MPD depends on μ only through the induced order, the order/magnitude distinction is the axis of Chapter 5.

3.4 Consistency and robustness

For a prediction-consuming algorithm, **consistency** is its ratio under perfect advice and **robustness** is its worst-case ratio under adversarial advice. Operationally we cannot sweep every adversarial prediction, so in the experiments robustness is reported as the minimum over our corruption levels, an upper bound on the true worst case and therefore a generous reading of an algorithm's safety. A useful algorithm is consistent and never (much) below ρ_{base}; the tension between the two is the subject of Chapter 6 and of the outlook in §10.2.

3.5 The experimental harness

The harness is a small, dependency-light Python codebase (NumPy, SciPy, NetworkX). Every source of randomness draws from its own independent, reproducible stream, so that within a comparison the only thing that varies is the prediction and adding an algorithm never disturbs existing results (Appendix A.1 lists the streams). We use **paired trials**: within a comparison, every algorithm and error level reuses the same graph, arrival sequence, OPT, and tie-break seed, so differences are attributable to the prediction alone. Reported ratios are means over trials with 95% normal-approximation confidence intervals, computed per algorithm and error level. Those per-algorithm intervals are the conservative choice: they under-use the pairing, since a comparison between two algorithms is really a *paired difference*, whose interval is narrower whenever the two respond to the same instances in the same direction. Every comparison the thesis draws survives under the wider convention (the narrowest margin being Feldman(MPD)'s +0.0044 against a ±0.0023 per-algorithm interval), so nothing in the thesis turns on which is used; §A.2 reports the measured correlations and paired intervals. Every figure and table in the thesis is regenerated from a fixed seed by a single script (Appendix A).

3.6 Fidelity check: reproducing Borodin et al.

Before building prediction-based algorithms on the harness, we validated it against the published experimental study of Borodin, Karavasilis and Pankratov [2]. This is a **fidelity check, not a contribution**: its purpose is to confirm that the generators, the i.i.d. sampler, the Hopcroft–Karp optimum, and the algorithm implementations reproduce the paper's qualitative findings, so that later differences can be attributed to the algorithms rather than to infrastructure. Our stack (Python/NetworkX) differs from the paper's (C++/Edmonds–Karp), so we target *qualitative* agreement, accepting small absolute differences (≤ 0.02).

We reproduced the core four algorithms (six greedy/non-greedy variants) on two random graph families at $n = 1000$, $m = n$, 100 trials per parameter value (matching the paper's setup), under a single master seed.

Erdős–Rényi (edge probability c/n; the paper's Fig. 9, partial). All five of the paper's

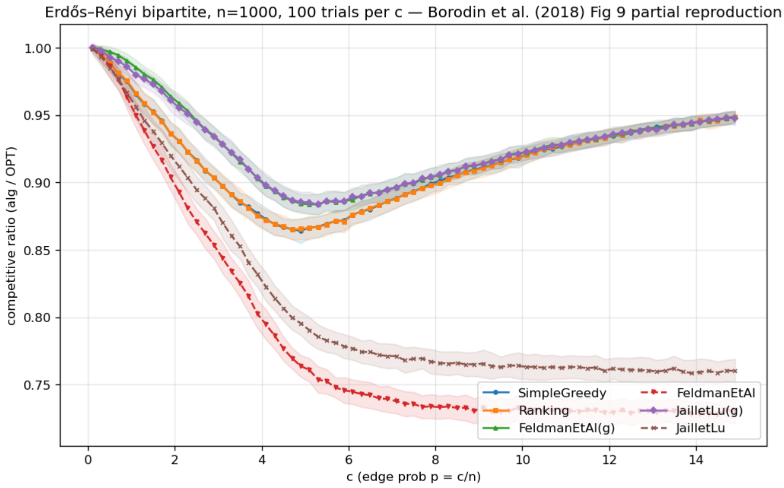


Figure 3.1: Erdős–Rényi reproduction: the characteristic U-shape, greedy minima near $c \approx 4.9$, non-greedy degradation as c grows.

qualitative claims that we checked reproduce, with absolute differences below 0.02. In detail: sweeping c reproduces the characteristic U-shape and its hard case, SimpleGreedy attains its minimum 0.864 at $c \approx 4.9$, and the greedy variants of the complex algorithms their minima ≈ 0.884 at $c \approx 5.3$. Ranking is indistinguishable from SimpleGreedy (maximum difference 0.0017 across all 75 values; the paper omits Ranking's curve for exactly this reason), and the non-greedy variants degrade monotonically as c grows ($0.986 \to 0.730$ for Feldman, $0.985 \to 0.760$ for Jaillet–Lu). Every one of the paper's five prose claims we checked holds (**Figure 3.1**).

Random left-regular (left-degree d; the paper's Fig. 18, partial; Figure 3.2). The pattern repeats with the hard case at $d = 5$ (SimpleGreedy minimum 0.890), Ranking $\approx$ SimpleGreedy (max difference 0.0013), non-greedy degradation as d grows, and greedy complex variants $\approx$ SimpleGreedy asymptotically.

A cross-family observation. Combining the two sweeps surfaces a non-obvious fact: the non-greedy Feldman and Jaillet–Lu algorithms converge to the same asymptotic ratio in both families (0.730 and 0.760 respectively, within 0.001 across families) and both sit above their worst-case theoretical bounds (0.670 and 0.729) by +0.06 and +0.03. The two families happen to converge to the same value here; we do not investigate whether that is universal. It is, in any case, an early and concrete instance of the thesis's recurring theme that the average case is far more

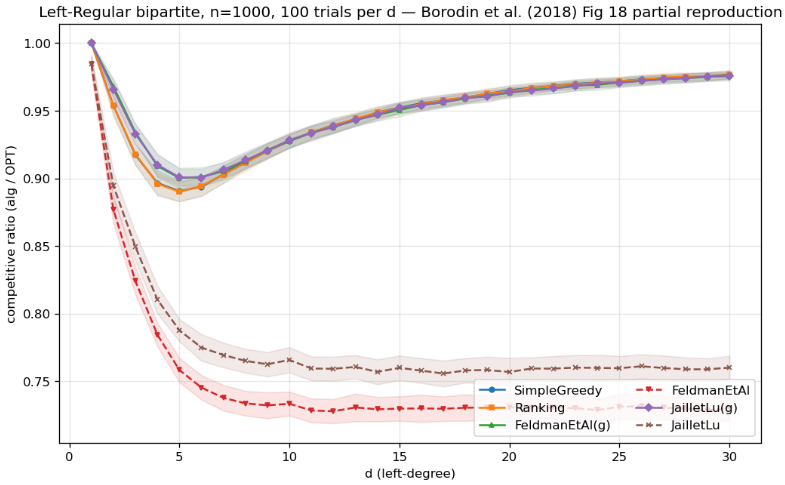


Figure 3.2: Random left-regular reproduction: the hard case at $d = 5$, Greedy $\approx$ Ranking.

benign than the worst case, which the prediction-error sweeps of later chapters probe in earnest.

Verdict. The harness reproduces the published qualitative findings on both families, including the locations of the hard cases and the near-identity of Greedy and Ranking. The foundation is trusted; the contributions of Chapters 4–9 are built on it. (Full reproduction tables and the paper-claim checklist are in Appendix A.)

4 The Unified Benchmark

Having validated the harness (Chapter 3), we now place all three algorithm families on it and establish the thesis's organizing finding. This chapter reports competitive ratios (mean $\pm$ 95% CI) as a function of prediction quality, in three panels chosen to span the regimes each family was designed for; the four findings it draws out (§4.2) recur through the rest of the thesis.

4.1 Design

The families consume different prediction objects, so each is driven by its own corruption knob and reported in a parallel panel (Chapter 3); the shared harness (graphs, OPT, CI methodology, and the advice-free floor) is what makes the panels comparable.

Instance format and notation. Every panel uses the instance format of Chapter 3: n offline resources, r online request types, and m arrivals drawn i.i.d. from the type distribution; throughout this chapter $m = n$, so requests and resources are balanced. The panels differ only in the type graph connecting requests to resources; that single change is what moves the input between the regimes the two prediction families were designed for. The three are chosen so that a degree predictor has strong signal, weak signal, and no role at all, respectively:

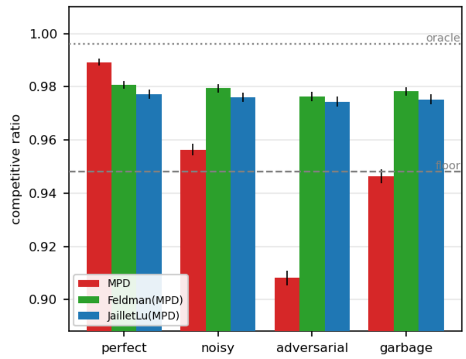
- **Panel A: heavy-tailed degrees (Zipf)** ($n = 1000$, 60 trials): resource degrees follow a Zipf power law with exponent 1.0, so a few resources are heavily contended while most are rarely eligible. This heavy-tailed profile gives a degree predictor genuine signal to carry.
- **Panel B: left-regular** d=5 ($n = 1000$, 60 trials): each arriving request connects to exactly $d = 5$ uniformly random resources, so resource degrees are nearly homogeneous: the hard case of Chapter 3, where a degree predictor has almost no signal left to carry.

- **Panel C: few-types** r=8 ($n = 2000$, 50 trials): only $r = 8$ distinct request types, each arriving $\approx n/r = 250$ times on average: the near-perfect-matchable, few-types regime the histogram-advice algorithms are calibrated for; their test-and-fallback test inspects a prefix of $k = 200$ arrivals.

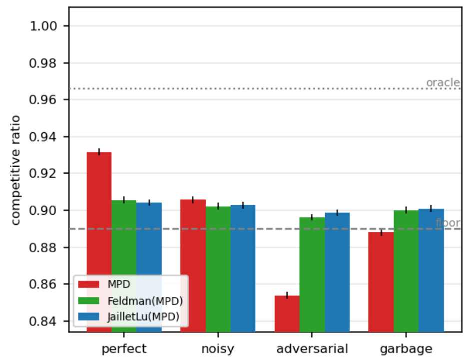
Panels A and B thus exercise the degree-prediction family (MPD and its augmentations); Panel C exercises the histogram-advice family (FollowPrediction, TestAndMatch, and the combiner).

Shared methodology. Paired trials, independent random streams and confidence intervals are exactly as in §3.5; the panel-specific parameters are the ones listed above, and the intervals are tight throughout (± 0.001–0.003). The quality columns instantiate the error models of §3.3. Degree panels: *perfect* (true realized degrees), *noisy* (random-flip at strength $\frac{1}{2}$), *adversarial* (order-reversing reflection), *garbage* (independent random μ, $\equiv$ Ranking); advice panel: the true histogram blended toward a concentrated random target by $\eta \in \{0, 0.3, 0.6, 1.0\}$ (*perfect / mild / bad / garbage*). The two sets of columns are not commensurable (they corrupt different prediction objects with different knobs), so only within-panel comparisons carry meaning. **Table 4.1** presents each panel's ratios beside its bar chart; the findings follow in §4.2.

Panel A — heavy-tailed (Zipf)	perfect	noisy	advers.	garbage
Ranking (floor)	0.948	—	—	—
MinDegree (oracle)	0.996	—	—	—
MPD	0.989	0.956	**0.908**	0.946
Feldman(MPD)	0.981	0.979	0.976	0.978
JailletLu(MPD)	0.977	0.976	0.974	0.975
Feldman (base)	0.887	—	—	—
JailletLu (base)	0.900	—	—	—



Panel B — left-regular d=5	perfect	noisy	advers.	garbage
Greedy = Ranking (floor)	0.890	—	—	—
MinDegree (oracle)	0.966	—	—	—
MPD	0.932	0.906	**0.854**	0.888
Feldman(MPD)	0.906	0.902	0.896	0.900
JailletLu(MPD)	0.904	0.903	0.899	0.901
Feldman (base)	0.760	—	—	—
JailletLu (base)	0.788	—	—	—



Panel C — few-types r=8	perfect	mild	bad	garbage
Ranking (floor)	0.990	—	—	—
MinDegree (oracle)	0.999	—	—	—
FollowPrediction	1.000	0.832	0.679	**0.472**
TestAndMatch (Choo)	1.000	0.984	0.989	0.990
TestAndMatch (BEM)	0.998	0.988	0.988	0.968
Combiner *(benchmark)*	0.990	0.990	0.990	0.990

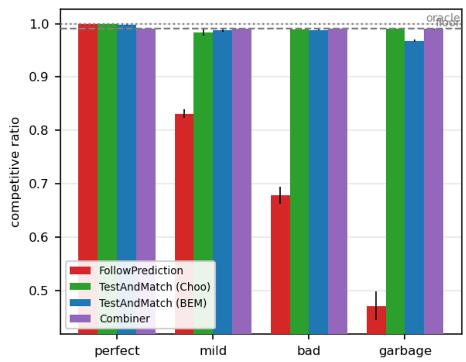


Table 4.1: The unified benchmark. Each panel's competitive ratios (means over paired trials; every 95% CI ≤ 0.003) sit beside their bar chart (error bars: 95% CIs; dashed line: advice-free floor; dotted: oracle ceiling). Bold marks the worst column of each unguarded prediction-follower; all three fall below their own panel's floor. That floor is instance-dependent — it is Ranking's ratio on that panel's graph family, not a constant — so the three floors (0.948, 0.890, 0.990) are not comparable with one another, and neither are the quality columns across the two prediction families (§3.3).

4.2 Four findings

(F1) Robustness is engineered, not free: naive followers crash below the floor. Both unguarded prediction-followers dive under the advice-free Ranking floor once the prediction is adversarial or garbage: MPD falls to $0.908 < 0.948$ (Panel A) and $0.854 < 0.890$ (Panel B), and FollowPrediction collapses to $0.472 \ll 0.990$ (Panel C). Under adversarial or garbage advice, then, using either *unguarded* is worse than using no prediction at all. Every robust algorithm (the augmentations, TestAndMatch, the combiner) avoids this by construction.

(F2) Two distinct robustness mechanisms, with different shapes. *Structural* robustness (Feldman(MPD), JailletLu(MPD)): the worst-case-optimal base matching carries the load and the prediction only breaks ties, so performance is nearly flat: Feldman(MPD) moves only $0.981 \to 0.976$ from perfect to adversarial (Panel A). It cannot crash but caps the upside (never reaching the 0.996 oracle). *Adaptive* robustness (TestAndMatch): test a sublinear prefix, then commit, capturing the upside when advice is good (Choo 1.000) and holding the floor when it is bad (0.990). On Panel C it is the only algorithm on the upper envelope at both ends. The two mechanisms trade consistency for robustness in opposite ways; **Figure 4.1** plots every algorithm on the consistency–robustness plane, where the opposite trades, and the empty region beyond TestAndMatch toward the ideal top-right corner, are visible at a glance.

(F3) The consistency upside is small on average-case inputs; the spread lives on the bad-advice side. On few-types the advice-free Ranking is already 0.990 and MPD-with-true-degrees is 0.999: the entire consistency headroom is 0.009 ± 0.001, smaller than most of the effects measured later in this thesis. Every wide gap in Panel C is a *downside* gap. This is the thesis in one panel; Chapter 6 shows why the affordable tests examined here struggle to capture the remaining upside, and the concluding outlook (§10.2) offers a theoretical interpretation of why this pattern may persist.

(F4) The augmentation rescues structurally weak base algorithms. Feldman and Jaillet–Lu are tuned for the worst-case ratio and are the weakest advice-free entries on these average-case inputs (Panel B: 0.760 / 0.788, below Greedy's 0.890); the MPD augmentation lifts them to ≈ 0.90. The prediction does more for the worst-case-designed algorithms than for greedy, a pairing that a unified table makes immediate.

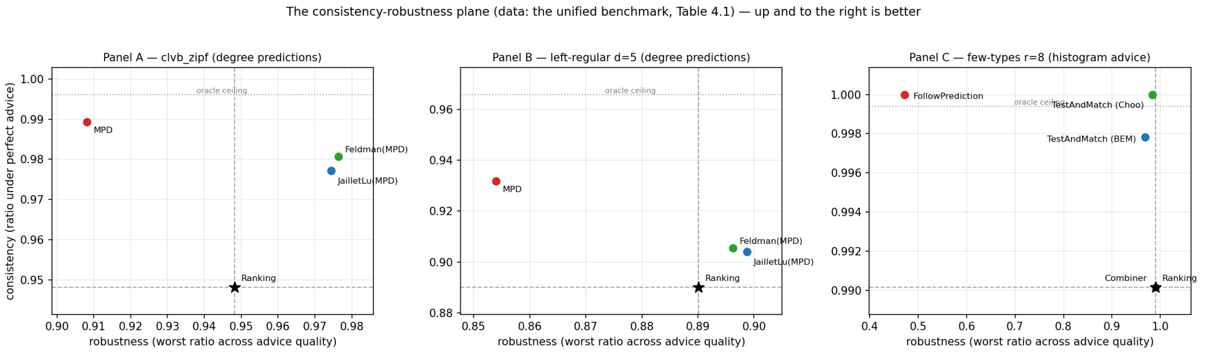


Figure 4.1: The consistency–robustness plane (the data of Table 4.1). Horizontal axis: ratio under perfect advice. Vertical axis: ratio under the worst corruption level, the operational robustness of §3.4. Dashed lines mark the advice-free floor, dotted the oracle ceiling; TestAndMatch sits nearest the ideal top-right corner.

4.3 Chapter summary

The value of a prediction here is downside protection, not performance. The next question is what governs the loss that remains, and Chapter 5 shows it is not the size of the prediction error but the order it induces.

5 What Governs the Loss: Order Error and What the Known Bound Does Not Say

Chapter 4 showed that on average-case inputs the loss of a degree-prediction algorithm is small. This chapter asks what *governs* that loss. MinPredictedDegree matches by ascending predicted degree, so it depends on the predictor μ *only through the order it induces* on the resources: two predictors inducing the same order produce identical matchings, and a monotone rescaling of μ changes nothing. The right question is therefore not "how large is the error" but "which *order* error governs the loss, and how tightly." Answering it requires care, because the qualitative fact that order, not magnitude, matters is already a theorem, and we are explicit about what is prior and what is ours.

5.1 What is already known (ACI)

Aamand, Chen and Indyk [1, Appendix D] prove that on the CLV-B model, MinPredictedDegree's matching loss relative to the true expected degrees is at most $n - \mathrm{LIS}(w[\mu])$. Here w is the vector of true expected degrees (written w rather than p to keep it distinct from the type distribution p of §3.1), $w[\mu]$ is that vector listed in the order μ induces, and LIS is the length of its longest non-decreasing subsequence: a pure order quantity. In particular a monotone (order-preserving) predictor leaves $w[\mu]$ already sorted, so $n - \mathrm{LIS} = 0$ and the loss is zero. Thus *order-dependence* and *the zero-effect of a monotone bias* are ACI's results, not ours; our systematic-bias error model (a monotone rescale) has Kendall-$\tau \equiv 0$ by construction and, consistently, leaves MPD's ratio exactly unchanged across the benchmark (Chapter 4): an empirical confirmation of ACI's statement, not a new finding.

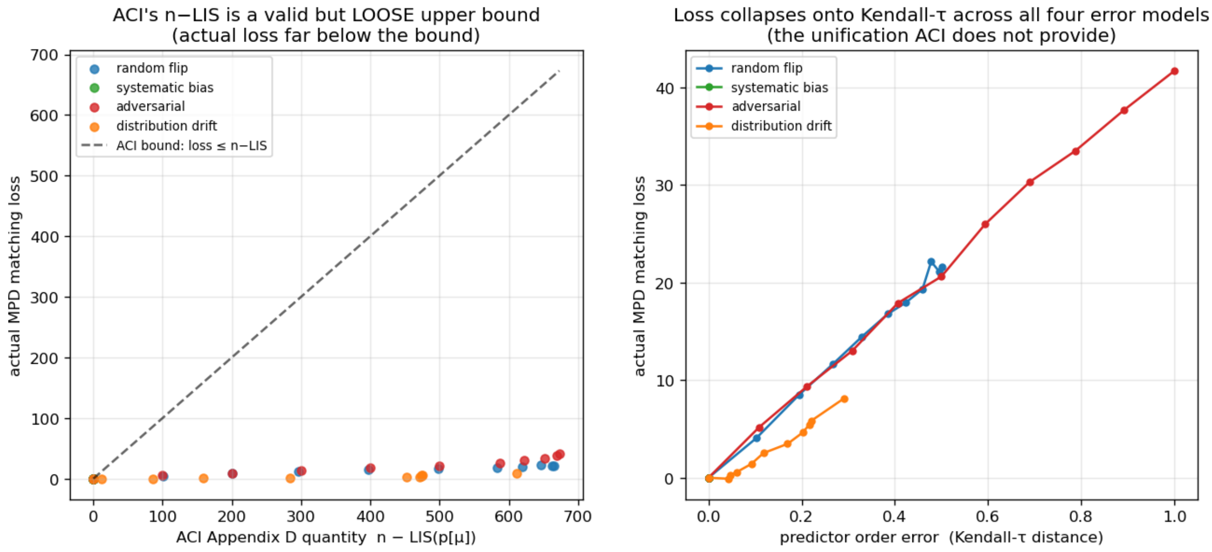


Figure 5.1: Order error governs the loss: the realized loss lies far below ACI's $n - \text{LIS}$ bound (a) and collapses onto Kendall-τ across all four error models (b).

5.2 Our characterization

We sweep the four structured error models across strength and record, per model and level on the same instances, three quantities: the actual MPD matching loss, ACI's $n - \text{LIS}$, and the normalized Kendall-τ order error, reported on $[0, 1]$, where 0 means the predicted order is exactly right and 1 that it is exactly reversed (**Figure 5.1**; $n = 1000$, Zipf exponent 1.0, 40 trials).

(i) ACI's $n - \text{LIS}$ bound is correct but very loose. The realized loss lies far below the bound at every point. Taking the ratio of the bound to the realized loss at each model's strongest corruption level, it is loose by roughly 16× (adversarial) to 75× (distribution-drift); all points hug the axis in Figure 5.1(a).

(ii) $n - \text{LIS}$ saturates and cannot distinguish error structures. For every non-trivial error it is pinned near its maximum (≈ 610–673 out of a possible $n = 1000$, against realized losses of 8 to 42), even though the true losses differ by a factor of five across models (8.1 drift, 21.6 random-flip, 41.7 adversarial). As a bound that is almost always $\approx n$, it carries little information about which prediction is more harmful.

(iii) Kendall-τ is the governing order measure, and the models collapse onto it. Plotted against Kendall-τ (Figure 5.1(b)), the four models fall on one increasing curve : loss rises with τ ($0.29 \to 0.50 \to 1.0$ for drift, random-flip, adversarial, tracking $8.1 \to 21.6 \to 41.7$), with systematic bias pinned at $\tau = 0$, zero loss. The collapse is not only visual: pooling all four models and all error levels (44 points), the rank correlation between Kendall-τ and the realized loss is Spearman $\rho = 0.979$ (Pearson $r = 0.992$), and within each non-degenerate model it is 0.97 or above. The quantity that predicts the loss and unifies the error structures is the Kendall-τ order distance, which the saturated $n - \text{LIS}$ cannot resolve.

5.3 Chapter summary

What this chapter adds, order-dependence itself being ACI's result (§5.1), is a precise empirical characterization of how much the known bound actually says: ACI's $n - \text{LIS}$ bound is loose by one to nearly two orders of magnitude and saturates, whereas Kendall-τ predicts the realized loss and unifies the four error structures. This both sharpens the theory's practical content and justifies the single order-error axis used when the predictor is stressed on real data. It is also why Chapter 7 can explain a cheap historical predictor's success by its order fidelity alone, and why Chapter 8 tried, and failed, to train a predictor for order directly.

6 Test-and-Fallback in Depth

The test-and-fallback algorithms are the adaptive robustness mechanism of Chapter 4. This chapter gives an empirical study in four parts: the robustness envelope they achieve (§6.1), a counter-intuitive failure of the acceptance threshold (§6.2), its recalibration (§6.3), and a benchmark of the dynamic combiner that explains the *commit-once* structure (§6.4). The resolution limit that recalibration exposes in §6.3 is this chapter's interface to the conclusion: it persists across the whole difficulty range, and §10.2 reads it quantitatively. Two naming conventions apply throughout. FollowPrediction is the unguarded follower, which mimics the advice matching without testing it; TestAndMatch is the test-and-fallback scheme in general, of which **Choo** and **BEM** are the two published instantiations. They share the test-then-commit structure and differ in how the acceptance threshold is set. The ℓ_1 test is the empirical surrogate the original authors also fall back to (Chapter 3).

6.1 The robustness envelope

Sweeping advice error $\ell_1(p, q)$ on few-types instances ($n = 2000$ arrivals, $r = 8$ request types, a tested prefix of $k = 200$ arrivals, 40 trials; **Figure 6.1**), FollowPrediction degrades linearly from 1.000 at perfect advice to 0.453 at $\ell_1 \approx 1.1$, well below the advice-free floor (≈ 0.99). TestAndMatch instead stays on the **upper envelope**: it captures the benefit when advice is good and, when advice is bad, its prefix test rejects it and it falls back to Ranking, never dropping below the advice-free floor at any point of this sweep (BEM 0.998 $\to$ 0.969; Choo 1.000 $\to$ 0.991). This is the adaptive counterpart of the structural robustness of Chapter 4.

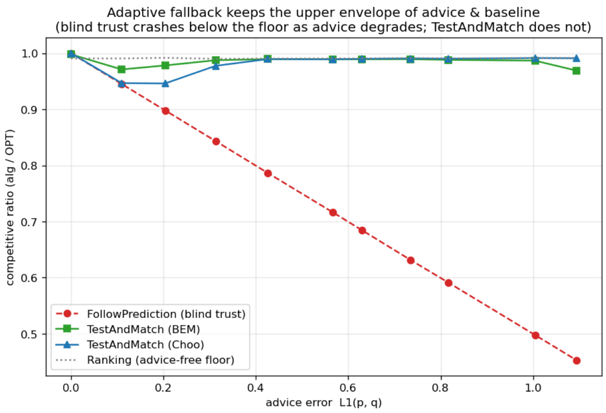


Figure 6.1: The robustness envelope: blind FollowPrediction crashes below the advice-free floor as advice error grows; TestAndMatch stays on the upper envelope.

6.2 A threshold that is too lenient on average-case inputs

Sweeping the prefix (testing) size k at *borderline* advice ($\eta = 0.15$, true $\ell_1 \approx 0.16$), where the test works hardest (**Figure 6.2**), a larger, more accurate test makes the *worse* decision: as k grows $25 \to 800$, the ratio falls $0.992 \to 0.956$ and the misjudgement rate rises $0.00 \to 0.60$.

The mechanism has two halves; the threshold is Choo and BEM's own, and what follows is our characterization of how it behaves off its design point. First, the acceptance threshold τ is calibrated to β, the competitive ratio the advice-free baseline is *proved* to achieve: here $\beta \approx 0.696$, Ranking's worst-case ratio under random arrival order, the value Choo et al. instantiate their threshold with [6]. On these instances the realized baseline is instead ≈ 0.99 (F3), so the empirical break-even sits at $\ell_1 \approx 0$, far below τ. Second, the prefix interacts with that gap in opposite directions at the two ends of the sweep. A small, noisy prefix over-estimates ℓ_1 and rejects the borderline advice, landing safely on the floor, but it is right for the wrong reason, rejecting because it is noisy rather than because the advice is bad. A large, accurate prefix measures $\ell_1 \approx 0.16 < \tau$ correctly and therefore accepts the mildly-bad advice that the worst-case threshold deems acceptable, underperforming the baseline. On strong-baseline inputs, in other words, the threshold is calibrated against the wrong baseline, and the size of the prefix decides only how faithfully the algorithm obeys it.

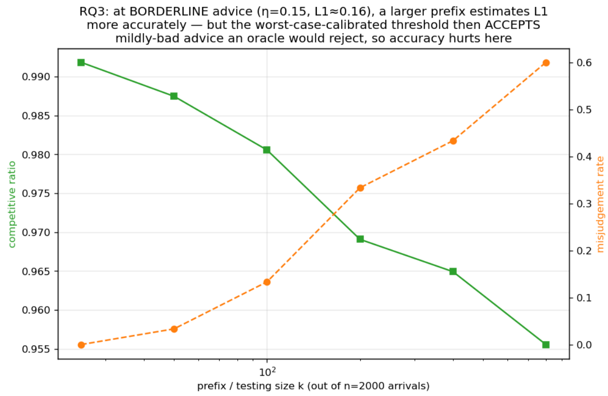


Figure 6.2: Testing cost at borderline advice: a larger, more accurate prefix test makes the *worse* decision under the worst-case-calibrated threshold.

6.3 Recalibration, and the resolution limit it exposes

By the **resolution** of a test we mean the smallest difference in ℓ_1 that its empirical estimator can distinguish from its own sampling noise at prefix length k; this section is about that quantity and the limit it imposes. Recalibrating τ to the *measured* baseline $\hat{\beta}$ eliminates the pathology (**Figure 6.3**): at borderline advice the worst-case threshold's misjudgement climbs $0.03 \to 1.00$ as the prefix grows and its ratio drops to 0.920, while the recalibrated threshold holds misjudgement 0.00 at every prefix and ratio ≈ 0.986. But recalibration exposes a deeper limit. At perfect advice the worst-case threshold scores 1.000 while the recalibrated one scores only 0.987: the recalibrated $\tau \approx 2(1 - \hat{\beta}) \approx 0.028$ is *smaller than the empirical-ℓ_1 estimator's noise floor*: the ℓ_1 distance between a length-k prefix's type frequencies and the true histogram under perfect advice, i.e. pure sampling noise, which falls as k grows and spans ≈ 0.13 down to ≈ 0.05 over the prefix lengths swept here, so it can never confidently *accept*: it rejects everything, including perfect advice, and always plays the baseline. In short:

> On strong-baseline instances, among thresholds of this form and at the prefix lengths we can afford, none both captures the consistency upside and stays safe: the upside is tiny and sits *below the estimator's resolution*. The worst-case threshold over-accepts; the recalibrated one over-rejects; a better tester only follows whichever threshold more faithfully.

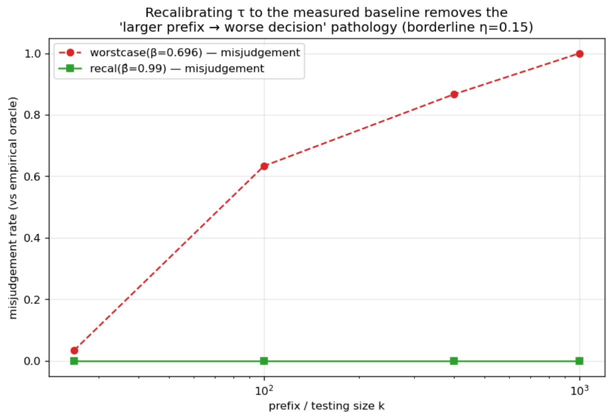


Figure 6.3: Recalibration removes the threshold pathology: misjudgement holds at zero at every prefix size, versus climbing toward 1.0 under the worst-case threshold.

Figure 6.4 widens this finding from one threshold to the whole difficulty range. Sweeping the number of types, the knob that sets the baseline strength, the *potential* upside of perfect advice grows as the baseline weakens, but the upside a sublinear-prefix test can *safely capture* stays pinned near zero: the empirical test's resolution (its noise floor) sits far above the break-even margin wherever an upside exists. The wall of this chapter is therefore not one badly calibrated threshold. The upside and the testing resolution move together, and the concluding outlook (§10.2) gives this coupling a quantitative form: the prefix needed to decide scales as the inverse square of the stakes.

6.4 The tested dynamic combiner is dominated, illustrating the case for test-then-commit

We benchmark the Chłędowski-style dynamic combiner to contextualize the commit-once structure of test-and-fallback. In its robust tuning the combiner sits exactly on the floor of the few-types family of §6.1 (0.990 across all advice quality): it never crashes but captures none of the consistency upside, strictly dominated by TestAndMatch. More instructively, an *eager* combiner that switches mid-stream reveals a penalty specific to irrevocable problems. On a smaller instance used as a mechanism check ($n = 600$, $r = 6$, a single seed with no averaging, one of the hand-verifiable

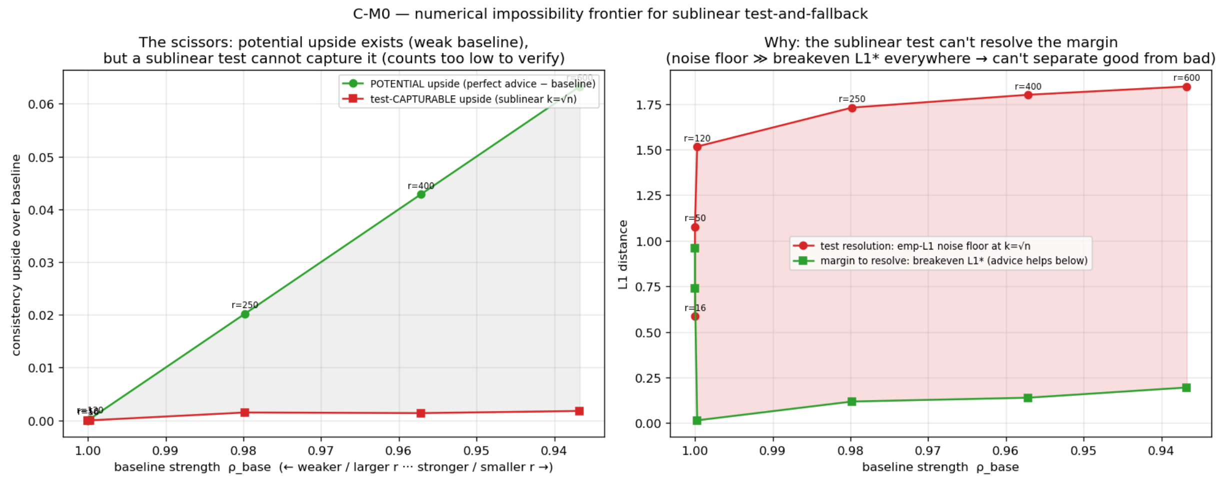


Figure 6.4: The testing-wall frontier. Horizontal axis: baseline strength ρ_{base}, swept by the number of request types (weaker baseline and larger r to the left). Vertical axis: consistency upside over the baseline. As the baseline weakens, the potential upside of perfect advice grows, but the upside captured safely by the sublinear tests examined remains near zero: at the tested points where upside exists, the empirical test's resolution lies above the break-even margin.

scripts of Appendix A.3), eager switching under perfect advice scores 0.927, below both the pure follower (1.000) and Ranking on that same instance (0.958), because switching from Ranking to advice-following mid-run lands the committed matching in an *incompatible hybrid*. Being one instance, this figure illustrates the mechanism rather than measuring its size; note also that its 0.958 is Ranking on that instance, not the 0.990 floor of the family above. In an irrevocable problem the follow/fallback decision must be made before the bulk of the commitments, which is why Choo/BEM test a prefix and then commit rather than switching dynamically. The dynamic combiner that is cheap insurance for caching does not port cleanly to matching.

6.5 Chapter summary

Test-and-fallback delivers the adaptive robustness envelope of §6.1, but its accept/reject decision is limited on the strong-baseline inputs tested here: none of the empirical-ℓ_1 thresholds examined both captures the upside and stays safe (§6.3), and the tested dynamic switching schemes are dominated by test-then-commit (§6.4). The resolution limit of §6.3, and its persistence across the tested difficulty range (Figure 6.4), is the empirical anchor of the theoretical outlook in §10.2; Figure 6.4 is the figure to remember from this chapter. The next chapter asks whether any of this survives outside synthetic instances.

7 External Validity

Chapters 4–6 use synthetic graphs and a synthetic error knob. Is the picture real? We stress it three ways, each replacing one synthetic ingredient with its real counterpart: §7.1 replaces the synthetic error with genuine temporal drift from a real request trace, §7.2 replaces the synthetic graphs with six real ones, and §7.3 replaces the hand-made predictor with a learned one.

7.1 A real, cheap predictor

We replace the synthetic knob with the cheapest realistic predictor: last-window historical statistics. Real Wikipedia daily pageviews give a live day (the truth) and earlier days (a 1-, 7-, or 30-day-stale forecast), so the error is genuine temporal drift; we map the trace onto a fixed serving topology and consume the forecast through the degree route (MPD) (**Figure 7.1**). Throughout this chapter we measure how much of the available benefit a predictor realizes by its **gap-capture**, $(\rho_{\mathrm{ALG}} - \rho_{\mathrm{base}})/(\rho_{\mathrm{oracle}} - \rho_{\mathrm{base}})$, the fraction of the baseline-to-oracle gap it closes. Three facts emerge.

First, **the predictor is cheap**. The with-predictions literature usually pictures an expensive learned model; here it is a linear-time count, about 0.11 ms per instance against 4.4 ms to compute OPT once, a few percent. These are the only wall-clock numbers in the thesis and are therefore machine-dependent; everything else is a ratio.

Second, **the benefit is real, partial, and never harmful**: a stale forecast reaches 27%–68% gap-capture (falling with staleness) and always stays above the baseline (0.938–0.957 vs 0.923–0.925, even at 30 days).

Third, and this is why: **topology aggregation makes the cheap predictor order-faithful**. The induced degree predictor's order error is only Kendall-$\tau \approx 0.19$–0.32, roughly half the raw

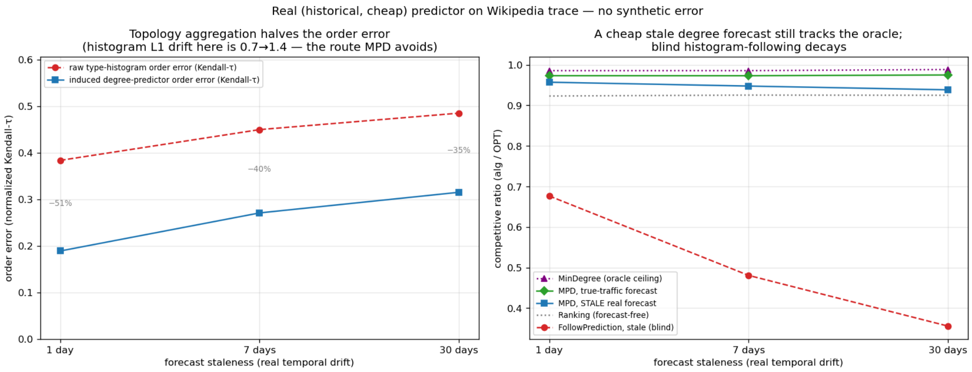


Figure 7.1: The figure to remember from this chapter: a real, cheap predictor (Wikipedia trace). The aggregated degree route survives staleness (b) because aggregation halves the order error (a); blind histogram-following decays.

histogram's (0.38–0.49), and since MPD depends only on order (Chapter 5), the aggregated route survives real drift. Consuming the *same* forecast through the raw histogram is catastrophic: blind FollowPrediction collapses to 0.68 $\to$ 0.36, far below the $\approx$ 0.92 baseline: exactly what the robust algorithms of Chapters 4 and 6 are for.

7.2 Six real-world graphs

We re-run the degree-prediction roster of Chapter 4 on the six Network-Repository graphs (two Facebook social, two C. elegans biological, two economic input-output), across the same quality columns, with 95% CIs (**Figure 7.2**). The two load-bearing findings hold across all six graphs tested. **F1 holds on all six**: naive MPD fed an adversarial predictor falls below the Ranking floor everywhere, by 0.06 (Reed98) to 0.10 (CE-PG). **F3 also holds on all six and is consistent with the proposed explanation**: the consistency upside is small on every graph (mean +0.049; range +0.022–+0.077) and smallest exactly where the baseline is strongest: the two dense economic graphs, with Ranking already 0.965/0.977, give the tiniest upsides.

The structural-robustness finding **F2 holds qualitatively on all six** (the augmentations' spread across quality columns is smaller than naive MPD's on every graph) and *strictly*, by the stronger criterion that the spread is at most half of MPD's, on the four social/bio graphs

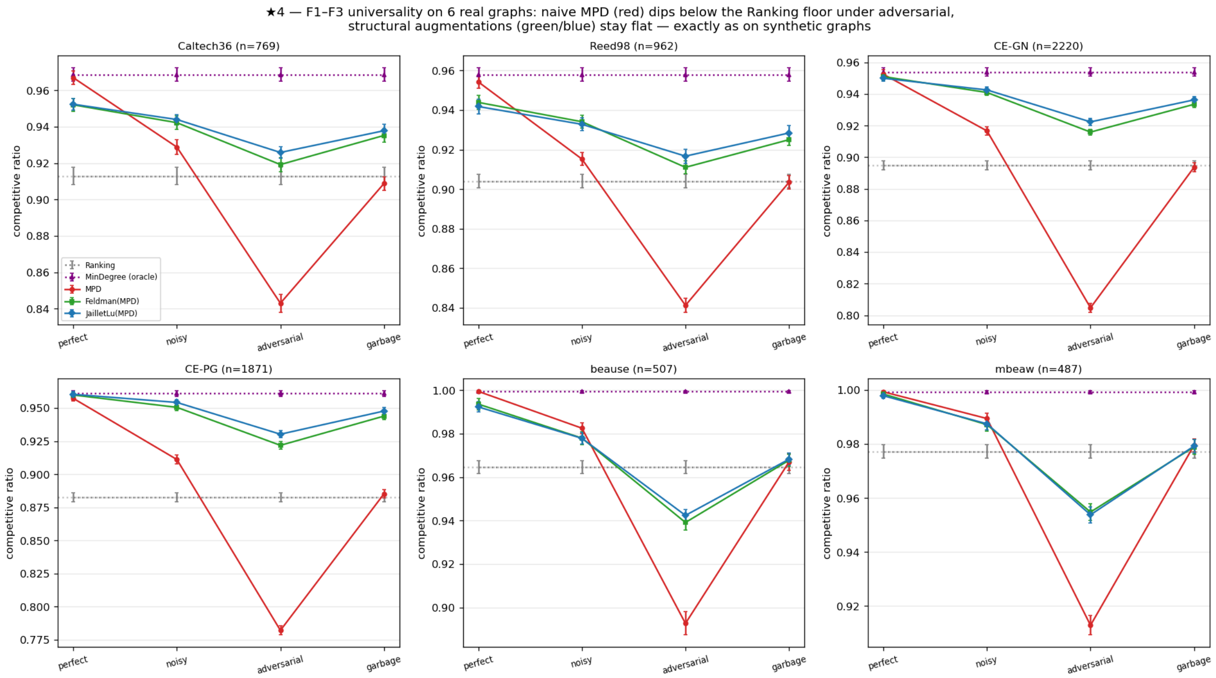


Figure 7.2: F1–F3 on six real graphs, one panel per graph (axis ranges differ between panels, as the graphs' floors do too, so read each panel against its own dashed floor rather than across panels). Naive MPD (red) dips below the Ranking floor everywhere; the structural augmentations (green/blue) stay flat.

(spread 0.22–0.29× MPD's); on the two dense economic graphs the protection is only *partial*: the augmentation cushions the adversarial drop (0.939 vs naive MPD's 0.893) but cannot clear the unusually high 0.965 floor, dipping ≈ 0.03 below it. Those two graphs are so dense that matching is nearly trivial (Ranking ≈ 0.97, MinDegree $= 1.00$), so there is neither upside to capture (F3) nor much downside to protect. This pattern is consistent with the mechanism underlying F3 rather than contradicting it. Finally, F4 is dramatic: the worst-case-designed Feldman/Jaillet–Lu are the weakest advice-free entries on the econ graphs (0.73–0.77) and the augmentation lifts them to 0.99, a +0.26 rescue.

7.3 Does learning the predictor help?

Because MPD consumes the predictor only through order (Chapter 5), one might train it with a rank loss rather than regression. We tested this and report an honest negative: the advantage is real on deliberately engineered features but disappears on real temporal ones, where rank- and regression-trained predictors induce the same order and reach the same ratio. The experiments, their numbers and the three-step argument behind them are in §8.1; what matters for external validity is only the consequence. Once a predictor is order-faithful, which a cheap historical count already is (§7.1), neither a better algorithm nor a better-trained predictor buys much on average-case matching.

7.4 Chapter summary

On the real predictors, real graphs, and learned predictor examined here, the same broad pattern recurs: the advice-free baseline is near-optimal, unguarded following is unsafe, and predictions buy downside protection rather than performance, with one instructive boundary: the two dense economic graphs of §7.2, where the augmentation cannot quite clear an unusually high floor. Having observed the same broad pattern across the settings examined, Chapter 8 reports the directions that tried to get past it, and the concluding outlook (§10.2) offers a theoretical account of why it may recur.

8 Exploratory Directions and Negative Results

The experimental chapters (4–7) establish a wall: on average-case matching the advice-free baseline is near-optimal, so predictions are robustness insurance rather than a performance lever. Before accepting the wall as final, we pursued three directions that each *tried to get past it*: learning the predictor to squeeze out more performance (§8.1), finding a serving regime where predictions genuinely help (§8.2), and letting a systematic literature review point us to the highest-value contribution (§8.3). All three produced results consistent with the same wall, and their convergence motivated the theoretical outlook of §10.2. Each is reported here with the evidence behind that conclusion.

8.1 Learning to rank the predictor (a negative result)

Chapter 5 shows that MPD consumes its predictor only through the order it induces. This suggests a concrete way to do better: rather than training a predictor to minimize its *regression* error to the true degrees (the standard "predict-then-optimize" objective), train it with an *order-aware* loss (a pairwise rank loss) so it optimizes the quantity the algorithm actually uses. We investigated this in three steps.

M1: the mechanism exists (**Figure 8.1**). With deliberately *divergent* synthetic features (one feature carrying magnitude, another carrying order), a rank-trained linear predictor sharply beats a regression-trained one on the decision metric: matching ratio 0.989 (essentially the oracle) versus 0.974, while the rank-trained predictor has *worse* regression error to the truth (MSE 87.6 vs 33.4) but better order (Kendall-τ 0.058 vs 0.255). The dissociation is real: the worse *fit* gives

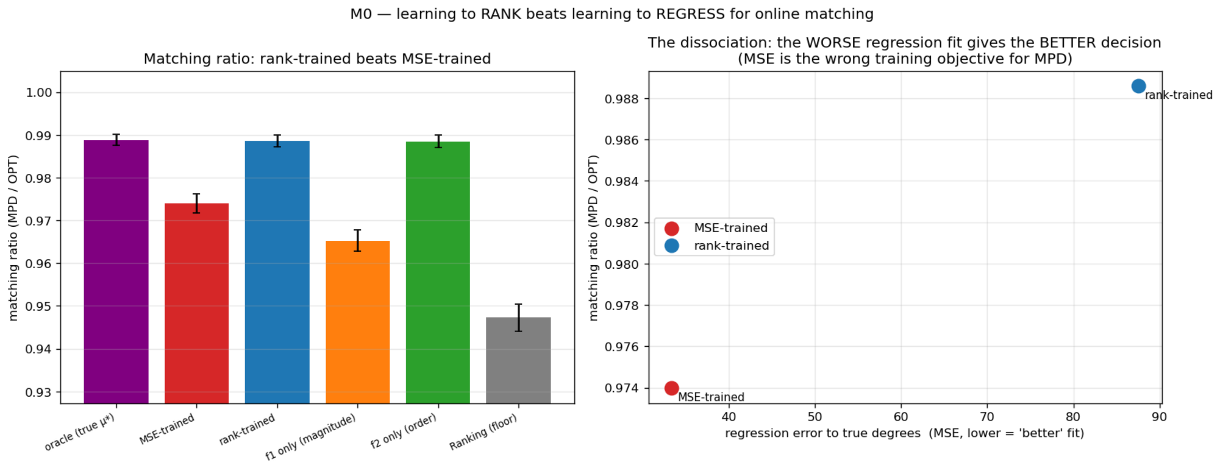


Figure 8.1: M1: with divergent features, rank-training beats regression on the matching ratio (left) despite a worse regression fit (right).

the better *decision*. Regression is thus the wrong training objective whenever the features separate magnitude from order, a condition M2 and M3 below show to be rare.

M2: the advantage is doubly gated, and small. Sweeping the feature divergence and the graph difficulty, the rank-advantage is zero when features do not induce an order/magnitude conflict, and zero on easy instances where the baseline is already optimal (the wall, again). It peaks at only +1.3% of the ratio on synthetic graphs; measured as gap-capture (§7.1) rather than as an absolute ratio, rank-training recovers essentially the full oracle gap while regression leaves about 30% of it unrealized, but the gap itself is small.

M3: it disappears on real features (**Figure 8.2**). The decisive test uses genuine temporal features from real serving traces (per-resource reference counts over the previous windows) to predict the next window (150 context-length types over 500 replicas of degree 8, 40 windows, three lag features, a 60/40 train/test split). Here the rank- and regression-trained predictors produce identical order (Kendall-τ 0.126 vs 0.126) and identical matching ratio. The order/magnitude divergence that powers rank-training arises in the *engineered* features used above. For the realistic lagged-count features examined here, the features appear to be collinear noisy estimates of the same popularity, and regression recovers the order as well as ranking does.

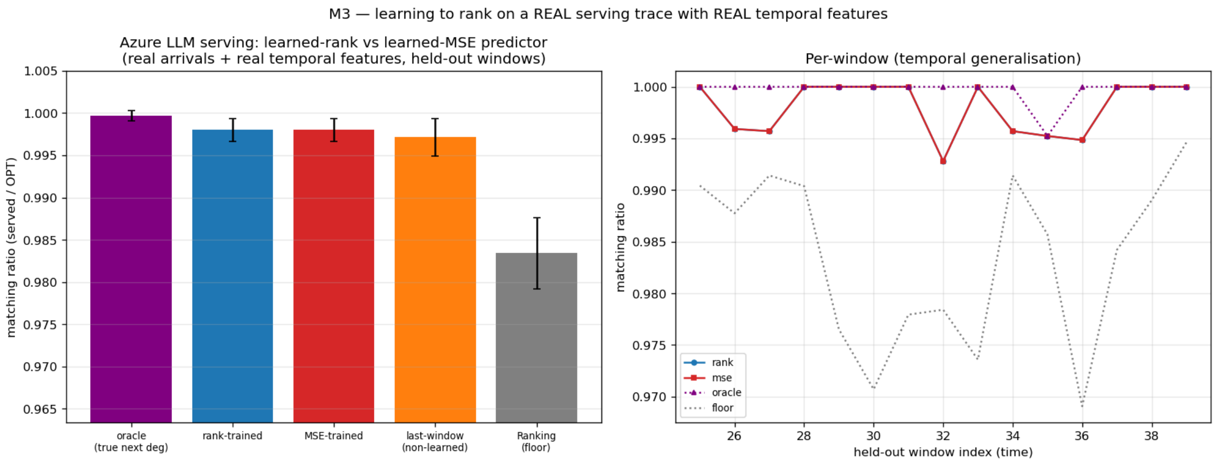


Figure 8.2: M3 (Azure trace, real temporal features): rank- and MSE-trained predictors are indistinguishable; the engineered divergence that powers rank-training does not arise in this experiment.

Verdict. Learning the predictor with a decision-aligned loss does not change the picture of Chapters 4–7, and we report it as a negative result: in these experiments, the win requires a feature divergence that did not arise on the real trace, and even where it does the payoff is bounded by the (small) baseline-to-oracle gap. The result is folded into the thesis as a negative that reinforces the central finding: once a predictor is order-faithful, which a cheap historical count already is (Chapter 7), neither a better algorithm nor a better-trained predictor buys much on average-case matching.

8.2 Rescuing the serving application with a with-predictions result (a negative probe)

The serving case study (Chapter 9) recovers established systems results and is therefore presented as a case study rather than a novelty claim. We asked whether a new actionable with-predictions result could rescue it. The escape, if one exists, must be a different *objective*, one on which the reactive baseline is genuinely far from optimal. The obstacle otherwise is again the wall: every

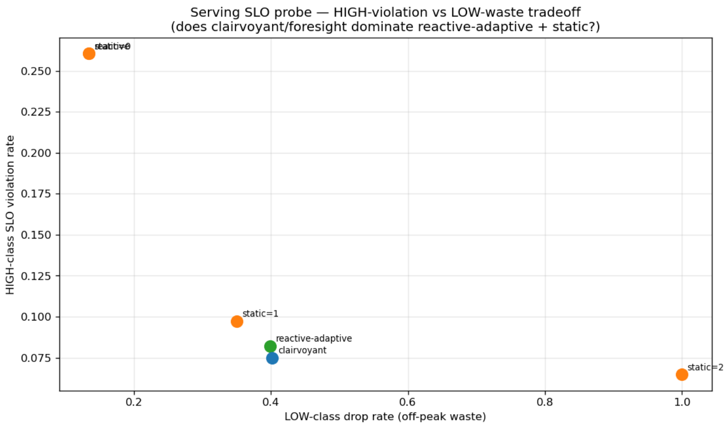


Figure 8.3: Serving SLO probe: the best non-predictive policy comes within a few percent of the clairvoyant reference, suggesting little additional value from foresight in this sweep.

serving variant we tried optimizes *throughput* (goodput), and throughput is forgiving, since under overload a reactive router fills capacity just as the optimum does.

We probed the most promising candidate: an **SLO / tail objective** (protecting a tight-SLO class of requests from being dropped) under bursty, non-stationary load, exactly the regime where a reactive policy, lacking foresight, might fail. Using an event-driven simulator we compared non-predictive policies (static capacity reservation; a reactive-adaptive policy that reserves based on *observed* recent load) against a **clairvoyant reference** that reserves based on the *actual future* burst. Two caveats belong with that design. The reference has perfect foresight but is not a proven optimum for the SLO objective, so it estimates what foresight is worth rather than bounding it; and the estimate is visibly not tight, because in the moderate regime a trivial static reservation of one slot drives tight-SLO violations to near zero and beats the clairvoyant reference outright, reserving for the actual future burst over-reserves there. What the sweep does establish is one-directional and still useful: across every regime (overload level, uniform vs bursty tight-SLO demand), the best non-predictive policy comes within $\leq 3\%$ (**Figure 8.3**) of a policy that knows the future exactly, so the particular thing a forecast would supply is not what these policies are missing. Two reasons, both robust to the sweep: protecting a tight-SLO minority needs only a small static headroom, no forecast; and bursts are persistent enough that reacting to observed load is almost as good as forecasting it.

Verdict. The tail objective is also forgiving in the regimes examined: a third face of the wall, after throughput (Chapters 4–7) and predictor-learning (§8.1). We did not identify a benefit from foresight within this sweep, so serving remains a case study. A regime that would break the wall (a non-stationary or adversarial objective where the baseline is far from optimal) is exactly the kind of setting the thesis brackets as future work.

8.3 A literature review that redirected the work

Deciding which of our findings were genuinely novel required a systematic prior-art review rather than intuition: we searched from several independent starting points and checked each candidate claim against the primary papers rather than their abstracts. The verdicts were sometimes deflating. We did not identify a prior unified benchmark or empirical study of test-and-fallback, so those results are worth leading with; the order-error finding is *partially* pre-empted by ACI's Appendix D and had to be reframed as a tightness/measure characterization rather than a discovery (Chapter 5); and the serving results are largely re-derivations of established systems facts, warranting their demotion to a case study (§8.2, Chapter 9). A focused second pass confirmed that no prior work quantifies the cost of the prefix test on strong-baseline instances, while flagging the one risk any such result must defend against (Choo et al.'s constructive baseline-coupling), an input to the outlook of §10.2.

The review is itself part of the research process reported here: it turned an undifferentiated pile of findings into a prioritized contribution, redirected effort away from low-value elaborations toward the theory question, and enforced the honesty guardrails carried throughout the thesis.

8.4 Synthesis: the negatives point to the same wall

Across the three explorations, a better-trained predictor did not help on the real features tested (§8.1), the predictions lens did not improve the tail objective examined (§8.2), and the literature review found no established route to an easy gain (§8.3). Together with Chapters 4–7, this recurrence motivates the structural interpretation considered in §10.2.

9 Application Case Study: AI-Inference Serving

The thesis's abstraction is not confined to classical matching markets; it instantiates a contemporary systems problem. This chapter casts request routing in an AI-inference serving system as online matching. It does two things. First, it tests whether the abstraction is rich enough to carry a live systems problem; recovering established results provides evidence that the mapping is faithful. Second, it supplies the setting in which Chapter 8 probed, and did not identify, a regime where predictions genuinely help. Because the systems facts recovered below are already known, the chapter is a **case study rather than a novelty claim**; that decision follows the prior-art review of Chapter 8.

9.1 The serving instantiation

We map serving to online **capacitated matching**: each offline resource is a model replica or cache shard that can serve up to c concurrent requests, so we match with capacity c rather than 1 (this is b-matching with every b equal to c; we write c throughout, including in the figures). Arrivals are requests drawn from a non-uniform, bursty traffic distribution over request types, and an edge is a capability or cache affinity. Two quantities must be kept apart. Raw **goodput** is the fraction of arriving requests that are served; the **competitive ratio** is the number served divided by the capacitated optimum on the same realized instance. They coincide only when the optimum can serve every arrival, which under overload it cannot, so we report the competitive ratio throughout. Figures 9.1 and 9.2 both plot it; Figure 9.3 measures a different objective entirely (the KV-cache hit fraction) and is labelled accordingly.

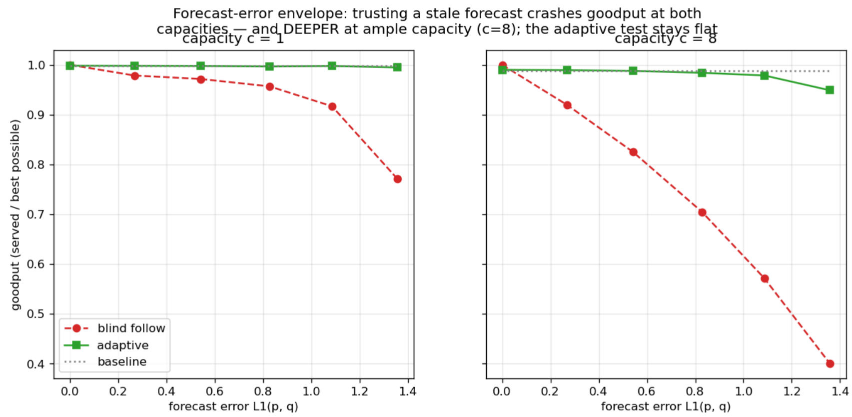


Figure 9.1: Capacity as robustness (synthetic serving topology: 200 replicas, 40 request types of degree 8, 800 arrivals, 25 trials per point): blindly following the forecast crashes the competitive ratio, deeper at ample capacity ($c = 8$), while the adaptive test stays flat.

For the dynamic experiment of Figure 9.2 that optimum is not a matching but a scheduling problem: admit a subset of the requests, each held on one compatible replica for its whole service time, at most c concurrent per replica. It is NP-hard in general, so we divide by a computable upper bound on it, obtained by relaxing the per-replica capacity to a per-type capacity $c \cdot \deg(\ell)$; the relaxed problem decomposes by type into interval scheduling and is then solved exactly. The reported ratios are therefore lower bounds on the true competitive ratio, and the bound is tight, since a feasible offline assignment brackets the optimum to within 1.1% of the arrival count at $c = 3$, 0.4% at $c = 6$, and exactly at $c = 12$. We exercise the instantiation on a synthetic serving topology (Figure 9.1, where the prediction quality can be swept directly) and on two real traces: an Azure LLM inference trace (Figure 9.2) and the Mooncake prefix-cache trace [18] (Figure 9.3). A third real trace, Wikipedia pageviews, drives the predictor study of §7.1.

Capacity as robustness (**Figure 9.1**). Increasing the capacity c smooths the effect of a bad prediction: a capacity-aware baseline stays safe, while blindly trusting a traffic forecast degrades further as capacity grows. Over the capacity range we swept, added capacity buys the same protection that the adaptive test does: the systems analogue of the robustness-insurance thesis, and a reminder that the cheaper of the two is a provisioning decision, not an algorithmic one.

Forecasts vs live load (**Figure 9.2**). Under dynamic service times (requests hold a slot for a

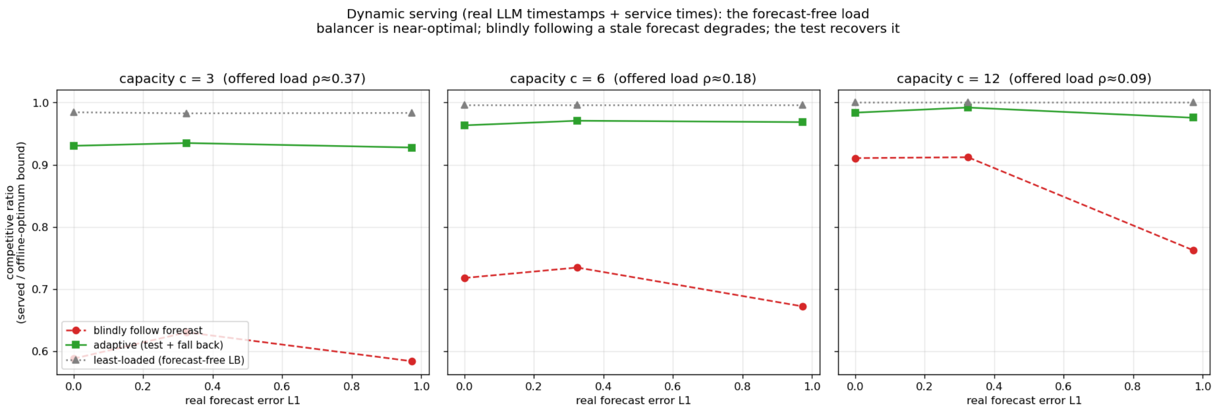


Figure 9.2: Live load beats a stale forecast under dynamic service times (Azure LLM trace, 9683 arrivals, 8 topologies per point): the forecast-free balancer is within 2% of the offline optimum at every capacity, blind following loses up to 40 points, and the prefix test recovers most of it. Ratios are against an upper bound on the offline optimum, hence conservative.

real duration, released event-by-event), a live-load signal beats a stale traffic forecast. Against the offline optimum the gap is stark: the forecast-free least-loaded balancer reaches 0.98–1.00 of the optimum at every capacity, while blindly following the forecast reaches only 0.58–0.91, and the prefix test recovers most of the loss (0.93–0.99) at the cost of the prefix it spends. The reactive policy is not merely better than the forecast-following one; it is within 2% of what perfect hindsight could have achieved.

Cache-affinity routing (**Figure 9.3**). For prefix-cache-aware routing, a *stable* affinity router beats a reactive one, the reverse of the traffic-forecast case, because cache locality rewards persistence [19].

Each of these recovers an established systems result cleanly, demonstrating that the framework instantiates the problem faithfully. They also line up into one observation: the first two say *react rather than forecast*, the third says the opposite, and the difference is the objective: cache locality rewards persistence where load balancing rewards responsiveness.

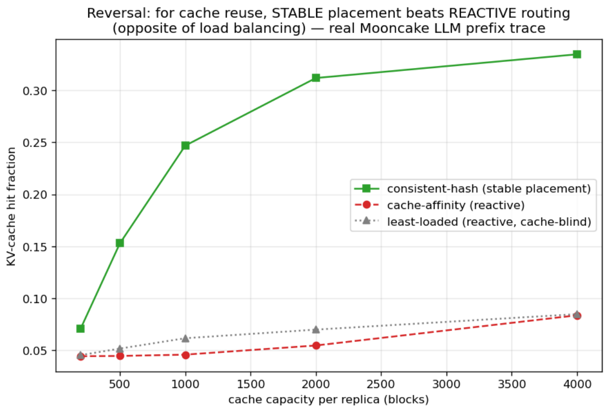


Figure 9.3: The cache-affinity reversal (Mooncake trace, 16 replicas, cache capacity swept from 200 to 4000 blocks): stable placement beats reactive routing for KV-cache reuse. The vertical axis is the KV-cache hit fraction, not a competitive ratio.

9.2 A probe for a genuinely new result, and its negative

We also asked whether the with-predictions lens yields a new actionable serving result on a tail rather than a throughput objective. In this probe it did not: on an SLO objective (protecting a tight-SLO class of requests under bursty load), a non-predictive policy comes within $\leq 3\%$ of one with perfect foresight across every regime swept. The experiment, its caveats and its two explanations are in §8.2. The consequence for this chapter is that we did not identify a benefit from foresight in the regimes examined, so serving remains a case study.

9.3 Chapter summary

The abstraction reaches a live systems problem and recovers its established results, and the tail-objective probe did not identify a genuine with-predictions win. What remains is to ask whether the wall these chapters keep meeting is an accident of our inputs or something forced: the question the conclusion takes up.

10 Conclusion and Future Work

10.1 Summary

This thesis studied learning-augmented algorithms for online bipartite matching by first building a common experimental foundation and then explaining what it revealed. On a single harness, validated against the published Borodin et al. study (Chapter 3), we compared the advice-free baselines, MinPredictedDegree and its augmentations, and the test-and-fallback algorithms, across synthetic graphs, six real-world graphs, and real request traces (Chapters 4–7).

Across the settings examined, the same broad picture appeared: the advice-free baseline is already near-optimal on these average-case inputs, so the consistency upside of a good prediction is small; unguarded prediction-following crashes below the baseline under bad predictions; and the value of the sophisticated algorithms is downside protection, delivered by either a structural or an adaptive robustness mechanism with distinct trade-offs.

Two sub-questions were sharpened along the way. The (small) loss is governed by a Kendall-τ order error rather than by magnitude, order-dependence itself being a prior result of Aamand, Chen and Indyk (Chapter 5). The adaptive test has a threshold-calibration pathology and a resolution limit (Chapter 6). We also report two directions that did not work: learning the predictor for extra performance, and finding a serving regime where predictions genuinely help (Chapter 8).

The recurring wall raises an obvious question: is it an accident of the inputs we chose, or is it forced? The next section gives our answer in outlook form. The single sentence that unifies the thesis is: **in the average-case online-matching settings examined, predictions act primarily as robustness insurance rather than as a performance lever, and their measured upside is smaller than the observed price of deciding whether to trust**

them.

10.2 A theoretical outlook: the price of testing advice

This section proves one inequality and then reads our own experiments through it; nothing beyond that inequality is claimed as a theorem. The experiments say that no *practical* acceptance threshold captures the upside safely (§6.3, Figure 6.4); the question here is whether a decision rule of some *other* kind could.

> **A trade-off inequality.** Let G and Bd be two instance distributions sharing the *same* advice, such that following the advice gains δ under G and loses Δ under Bd relative to the advice-free baseline, and let γ_k be the total-variation distance between the laws of their length-k prefixes, equivalently the largest possible difference between their acceptance probabilities for any test that sees only the first k arrivals. Then every test-and-fallback algorithm, deciding by *any* measurable rule on its prefix, satisfies
>
> $$(1-\eta_c) \;\le\; \eta_r + \gamma_k + o(1),$$
>
> where η_c is the fraction of the upside it forgoes under G and η_r its robustness loss under Bd as a fraction of Δ.

The proof is a short conditioning argument: capturing the upside forces the algorithm to follow under G, robustness forces it to fall back under Bd, and no function of the prefix can behave differently on two prefix distributions that are statistically γ_k-close. "Consistent and robust" thus becomes a question of *sample complexity*: how long a prefix is needed to tell advice worth following from advice worth rejecting?

On the rare-resource instances that produce Figure 6.4 (a *decomposable* family, in which the instance splits into independent cells that the advice acts on separately), the answer takes the form of a **budget–stakes** relation. Write δ for the *stakes*: what good advice gains over the baseline. Under a budget law that this thesis does not prove, the decision costs a prefix of $k^* = \tilde{\Theta}(\theta/\delta^2)$, the inverse square of the stakes, scaled by a contention parameter θ of the same order as the baseline slack $1-\rho_{\text{base}}$, and is met by a simple *directional* statistic (do the prefix arrivals agree with the advice more often than they contradict it?). The stakes are themselves capped by that slack, since advice can only win what the baseline leaves on the table. Under this unproved budget-law interpretation, substituting predicts that an upside below $\Theta\big(\sqrt{(1-\rho_{\mathrm{base}})/n}\big)$ would remain beyond the reach of a prefix-based decision rule with $k \le n$ on this decomposable instance family. At Chapter 4's parameters ($\rho_{\mathrm{base}} \approx 0.99$, $n = 2000$), the predicted scale is ≈ 0.004, the same order as the upsides measured in F3. The empirical wall is therefore consistent with this reading: potential and capturable upside separate as the baseline strengthens because the stakes shrink faster than the test's resolution improves. Verifying the prediction, including its scope beyond this family, remains future work.

Two notes bound this. The relation cuts both ways: where the stakes are large (a weak baseline) the directional statistic is cheap and following good advice is easy, so the wall is a statement about strong-baseline average-case inputs, not about testing in general; and it follows from the stakes being small rather than from distribution testing being expensive, the ℓ_1-threshold blindness of §6.3 coming from testing the *distance* rather than the *payoff*. And only the displayed inequality is claimed here: the budget law itself, and whether it extends beyond decomposable families, is not established in this thesis (§10.4, §10.5).

10.3 Critical evaluation

The three objectives of §1.2 were met to different degrees. Q1 is answered experimentally: Chapter 4 compares both algorithm families on one harness, and Chapter 7 checks that the main ordering persists with real predictors and graphs. Q2 is also answered, with an important qualification. Chapter 6 identifies the testing cost, threshold-calibration pathology, resolution limit, and irrevocability penalty of the published empirical-ℓ_1 mechanism; these findings do not rule out every possible prefix test. Q3 receives a bounded answer: §10.2 proves one trade-off inequality and gives a quantitative interpretation consistent with the measured scale, but does not prove that the wall is unavoidable.

Two design choices were especially valuable. Structurally targeted error models exposed the distinction between magnitude and order error, while validating the harness against Borodin et al. before extending it made later comparisons credible. Restricting the theoretical claim to the inequality actually proved also keeps the conclusion commensurate with the evidence.

The main trade-offs are equally clear. The known-i.i.d. model enables a clean comparison but also favours a strong advice-free baseline; matching size alone leaves open objectives such as tail latency and fairness; and the Python harness limited experiments to roughly $n = 2000$, where the scaling interpretation of §10.2 is only weakly tested. The learning-to-rank study also suggests a practical process lesson: testing realistic features before elaborating synthetic mechanisms would have reached the negative conclusion more quickly.

Overall, two objectives are met experimentally and the third in bounded form. The recurring wall is well supported across the settings varied here, but its survival under other objectives or non-average-case arrival orders remains uncertain.

10.4 Limitations

- **Input model.** We work in the known-i.i.d. model. Because every known-i.i.d. instance is also a random-order instance, guarantees proved in the random-order model carry over to ours (but not conversely, §2.1). The empirical wall is an *average-case* statement; we do not claim it for adversarial arrival order.
- **Test model.** Following the original authors, the test-and-fallback experiments use an empirical-ℓ_1 surrogate for the (unimplemented) distribution tester; §10.2 offers a structural interpretation of why the surrogate becomes blind, rather than establishing that claim as a separate theorem.
- **Prediction-object heterogeneity.** The degree- and histogram-prediction families do not map onto every graph, which is why they are reported in parallel panels rather than one table.
- **Data breadth.** Each real modality is exercised by one trace.
- **Objective.** We evaluate matching size (goodput) only. On objectives where the advice-free baseline is not near-optimal (tail latency, per-type fairness, migration or recompute cost), the picture could differ; our probe in §8.2 does not close that space.
- **Theory scope.** The thesis deliberately confines its theory to the outlook of §10.2: one proved trade-off inequality and a quantitative reading of the experiments. The full budget–

stakes law is companion work in preparation, and no theorem beyond the stated inequality is claimed here.

10.5 Future work

The thesis brackets, rather than resolves, the settings in which predictions might genuinely help online matching, and these are the natural next directions.

- **Beyond average-case inputs.** Adversarial or non-stationary arrivals, where the baseline can be far from optimal, may reveal a positive role for predictions.
- **Beyond throughput.** Tail latency, fairness, migration, or recomputation cost may admit gains that matching size does not; the SLO probe of Chapter 8 tests only one such case.
- **Completing the theory.** A sharp two-sided budget–stakes law, its attainable test, and its scope beyond decomposable families remain to be established. Tests that reuse online decisions as samples are another practical direction.
- **The full streaming and offline arm.** The ladder of Appendix A.7 is deliberately small. Completing it — memory accounting, the $(1 - \varepsilon)$ multi-pass approximations, an adversarial edge order — would test whether revision keeps beating advice once the streaming algorithm is the strong one rather than the simplest one.

10.6 Closing

Predictions are a powerful tool for online algorithms, but this thesis is a study of their *limits* on one well-understood problem. In the average-case settings examined here, the evidence supports three conclusions. A cheap, order-faithful predictor captures most of the small improvement available over the baseline. The sophisticated machinery serves mainly as insurance rather than as a source of further performance. On the tested inputs, deciding whether to trust a prediction appears to cost more than the remaining improvement is worth. Recognizing where predictions cannot help is, we hope, as useful as knowing where they can.

Bibliography

[1] Anders Aamand, Justin Y. Chen, and Piotr Indyk. 2022. (optimal) online bipartite matching with degree information. In *NeurIPS*.

[2] Allan Borodin, Christodoulos Karavasilis, and Denis Pankratov. 2020. An experimental study of algorithms for online bipartite matching. *ACM Journal of Experimental Algorithmics*, 25.

[3] Kunanon Burathep, Thomas Erlebach, and William K. Moses Jr. 2026. Learning-augmented online bipartite matching in the random arrival order model. In *Proceedings of the 51st International Conference on Current Trends in Theory and Practice of Computer Science (SOFSEM)*.

[4] Clément L. Canonne, Ayush Jain, Gautam Kamath, and Jerry Li. 2022. The price of tolerance in distribution testing. In *COLT*. arXiv:2106.13414; tolerant identity testing = $\tilde{\Theta}(n/\log n)$ at constant tolerance.

[5] Jakub Chłędowski, Adam Polak, Bartosz Szabucki, and Konrad Żołna. 2021. Robust learning-augmented caching: an experimental study. In *ICML*.

[6] Davin Choo, Themis Gouleakis, Chun Kai Ling, and Arnab Bhattacharyya. 2024. Online bipartite matching with imperfect advice. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

[7] Joan Feigenbaum, Sampath Kannan, Andrew McGregor, Siddharth Suri, and Jian Zhang. 2005. On graph problems in a semi-streaming model. *Theoretical Computer Science*, 348, 2–3, 207–216.

[8] Jon Feldman, Aranyak Mehta, Vahab Mirrokni, and S. Muthukrishnan. 2009. Online stochastic matching: beating 1-1/e. In *FOCS*.

[9] John E. Hopcroft and Richard M. Karp. 1973. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*.

[10] Patrick Jaillet and Xin Lu. 2014. Online stochastic matching: new algorithms with better bounds. *Mathematics of Operations Research*, 39, 3, 624–646.

[11] Jiantao Jiao, Yanjun Han, and Tsachy Weissman. 2018. Minimax estimation of the L_1 distance. *IEEE Transactions on Information Theory*, 64, 10, 6672–6706.

[12] Richard M. Karp, Umesh V. Vazirani, and Vijay V. Vazirani. 1990. An optimal algorithm for on-line bipartite matching. In *STOC*.

[13] Thodoris Lykouris and Sergei Vassilvitskii. 2018. Competitive caching with machine learned advice. In *ICML*.

[14] Vahideh H. Manshadi, Shayan Oveis Gharan, and Amin Saberi. 2012. Online stochastic matching: online actions based on offline statistics. *Mathematics of Operations Research*, 37, 4, 559–573.

[15] Andrew McGregor. 2005. Finding graph matchings in data streams. In *APPROX-RANDOM*, 170–181.

[16] Michael Mitzenmacher and Sergei Vassilvitskii. 2020. Algorithms with predictions. In *Beyond the Worst-Case Analysis of Algorithms*. Tim Roughgarden, (Ed.) Cambridge University Press, 646–662.

[17] Liam Paninski. 2008. A coincidence-based test for uniformity given very sparsely sampled discrete data. *IEEE Transactions on Information Theory*, 54, 10, 4750–4755.

[18] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: a kvcache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079*.

[19] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2024. Preble: efficient distributed prompt scheduling for llm serving. *arXiv preprint arXiv:2407.00023*.

[20] Gregory Valiant and Paul Valiant. 2011. Estimating the unseen: an $n/\log n$-sample estimator for entropy and support size, shown optimal via new clts. In *STOC*.

[21] Gregory Valiant and Paul Valiant. 2017. An automatic inequality prover and instance optimal identity testing. *SIAM Journal on Computing*, 46, 1, 429–455.

[22] Alexander Wei and Fred Zhang. 2020. Optimal robustness-consistency trade-offs for learning-augmented online algorithms. In *NeurIPS*.

[23] Toru Yoshinaga and Yasushi Kawase. 2026. Analyzing the effect of prediction accuracy on the distributionally-robust competitive ratio. *arXiv preprint arXiv:2601.06813*.

Reproduction Guide

Every quantitative result in this thesis is regenerated from a fixed seed by a single script. Two classes of script must be distinguished. Most are *self-contained*: they generate their own synthetic instances and run as-is on a clean checkout. The rest (the real-graph and real-trace results of Chapters 7, 8 and 9) first need external data placed under `data/`; that data is not redistributed with the code, and §A.5 records what each dataset is and where it comes from. In the map below, a dagger (†) marks the scripts of the second class.

This appendix maps each figure and table to its script, gives the commands and runtimes, lists the full Phase-2 reproduction tables deferred from Chapter 3, and records data provenance.

1 Environment and principles

- **Stack:** Python 3.12; NumPy 1.26+, SciPy 1.13, NetworkX 3.3, Matplotlib (plots only).
- **Determinism:** all randomness derives from one master seed (default `0`) via NumPy's `default_rng(seed).spawn(k)`, which yields independent, reproducible streams (graph, instance, algorithm randomness, prediction perturbation). Results are bit-stable across NumPy minor versions from 1.25 (BitGenerator-based spawning).
- **Paired trials:** within a comparison, every algorithm and error level reuses the same graph, arrival sequence, `OPT`, and tie-break seed, so differences are attributable to the prediction alone.
- **Confidence intervals:** 95% normal-approximation half-widths over trials.
- **Flow decompositions.** Three preprocessing steps (Feldman, Jaillet–Lu, and the serving advice b-matching) take a maximum flow and consume its *decomposition*, which, unlike the flow value, is not unique. Their networks label nodes with integers rather than strings,

so that the decomposition NetworkX returns is stable from run to run; with string labels it followed Python's per-process string hashing and every number derived from it moved between runs of the same script.

- Each script writes `results/<name>.json` (raw means/CIs) and `results/<name>.png` (plot), and prints per-step progress.

2 Figure / table → script map

Thesis object	Script	Output	≈ runtime
Fig 3.1 (ER U-curve)	`scripts/run_er_full.py`	`results/er_full.{json,png}`	~20 min
Fig 3.2 (left-regular)	`scripts/run_left_regular.py`	`results/left_regular.{json,png}`	~10 min
§3.1 / §3.5 methodology checks	`scripts/run_metric_check.py`	`results/metric_check.json`	~90 s
Table 4.1 (unified benchmark)	`scripts/run_unified_benchmark.py`, then `plot_unified_panels.py`	`results/unified_benchmark.{json,png}`, `unified_benchmark_panel{A,B,C}.png`, `unified_benchmark_tables.md`	~100 s
Fig 4.1 (consistency–robustness plane)	`scripts/run_consistency_robustness.py`	`results/consistency_robustness.{json,png}`	~1 s
Fig 5.1 (order-error vs ACI)	`scripts/run_order_vs_theory.py`	`results/order_vs_theory.{json,png}`	~30 s
Fig 6.1 (envelope), Fig 6.2 (prefix sweep)	`scripts/run_choo_bem.py`	`results/choo_bem_{envelope,prefix}.png`	~20 min
Fig 6.3, recalibration (§6.3)	`scripts/run_recalibration.py`	`results/recalibration_*.png`	~1.5 min
Fig 6.4 (testing-wall frontier)	`scripts/run_impossibility_frontier.py`	`results/impossibility_frontier.{json,png}`	~6 s
Fig 7.1 (real predictor) †	`scripts/run_real_predictor.py`	`results/real_predictor.{json,png}`	~15 s
Fig 7.2 (six real graphs) †	`scripts/run_realworld_robustness.py`	`results/realworld_robustness.{json,png}`	~65 s
Fig 8.1 (M1 rank vs MSE)	`scripts/run_rank_vs_mse_mve.py`	`results/rank_vs_mse_mve.{json,png}`	~10 s
M2 sweep (§8.1, no figure)	`scripts/run_rank_when_it_matters.py`	`results/rank_when_it_matters.{json,png}`	~20 s
Fig 8.2 (M3 real-trace learning) †	`scripts/run_rank_real_trace.py`	`results/rank_real_trace.{json,png}`	~10 s
Fig 8.3 (serving SLO probe)	`scripts/run_serving_slo_probe.py`	`results/serving_slo_probe.{json,png}`	~1 s
Figs 9.1–9.3, serving (Ch 9) †	`scripts/run_serving.py`, `run_serving_trace.py`, `run_serving_dynamic.py`, `run_prefix_cache.py`	`results/serving_*.png`, `prefix_cache_*.png`	varies
Ladder, streaming (§A.7)	`scripts/run_streaming_ladder.py`, then `plot_streaming_ladder.py`	`results/streaming_ladder.{json,png}`, `streaming_ladder_tables.md`	~40 s

Thesis object	Script	Output	≈ runtime
Outlook checks (§A.6)	`scripts/verify_witness_gap.py`	console output	seconds
Real-world Borodin Tables 3/4 (validation) †	`scripts/run_realworld.py`	`results/realworld.json`	~few min

Runtimes are wall-clock on one machine (Apple M4 Pro, 12 cores); the scripts are single-threaded apart from NumPy's own vectorization, so they scale with single-core speed.

Interval convention (deferred from §3.5). `run_metric_check.py` also compares the per-algorithm confidence intervals the thesis reports against paired-difference intervals on the same trials. Where the per-instance ratios are positively correlated the paired interval is 1.7–2.6× narrower (correlations 0.67 and 0.85); where they are not (blind following against Ranking under garbage advice, correlation −0.15) pairing does not help and the per-algorithm interval is the honest one. The narrowest margin in the thesis, Feldman(MPD)'s +0.0044 against a ±0.0023 per-algorithm interval, tightens to ±0.0009 under pairing.

3 Reproduction commands

```
# from the project root (matching-experiments/)
# correctness anchors: 8 hand-verifiable test files, all pass:
for t in tests/test_*.py; do python3 "$t"; done

# regenerate the headline results (fast ones):
python3 scripts/run_unified_benchmark.py        # Table 4.1 (data)
python3 scripts/plot_unified_panels.py          # Table 4.1 (panel charts; needs the line above)
python3 scripts/run_order_vs_theory.py          # Fig 5.1
python3 scripts/run_real_predictor.py           # Fig 7.1
python3 scripts/run_realworld_robustness.py     # Fig 7.2
python3 scripts/run_impossibility_frontier.py   # Fig 6.4
python3 scripts/run_rank_vs_mse_mve.py          # Fig 8.1
python3 scripts/run_rank_when_it_matters.py     # M2 (§8.1)
python3 scripts/run_rank_real_trace.py          # Fig 8.2
python3 scripts/run_serving_slo_probe.py        # Fig 8.3
python3 scripts/run_consistency_robustness.py   # Fig 4.1 (needs run_unified_benchmark first)
python3 scripts/run_metric_check.py             # §3.1 and §3.5 methodology checks
python3 scripts/run_streaming_ladder.py         # §A.7 ladder (data)
python3 scripts/plot_streaming_ladder.py        # §A.7 figure (needs the line above)

# the long (max-flow / Hopcroft-Karp-bound) sweeps:
python3 scripts/run_er_full.py                  # Fig 3.1  (~20 min)
python3 scripts/run_left_regular.py             # Fig 3.2  (~10 min)
python3 scripts/run_choo_bem.py                 # Fig 6.1, 6.2 (~20 min)
```

All scripts hardcode seed `0` unless noted; re-running reproduces the reported numbers.

4 Full Phase-2 reproduction tables (deferred from §3.6)

Setup: $n = 1000$, $m = n$, 100 trials per parameter value, seed 0. Target is *qualitative* agreement with Borodin et al. (Python/NetworkX vs the paper's C++/Edmonds–Karp; absolute differences ≤ 0.02 accepted).

Erdős–Rényi, competitive ratio at selected c (SG=SimpleGreedy, Rk=Ranking, F/J = Feldman/Jaillet–Lu, -NG/-G = non-greedy/greedy):

c	SG	Rk	F-NG	F-G	J-NG	J-G
0.10	1.000	0.999	1.000	1.000	0.999	1.000
1.90	0.936	0.936	0.904	0.964	0.920	0.961
4.90	**0.864**	0.866	0.764	**0.884**	0.795	**0.886**
8.90	0.909	0.909	0.730	0.912	0.765	0.913
14.90	0.949	0.949	0.730	0.949	0.760	0.948

Per-algorithm minima (ER): SG 0.864 @ c=4.9; Rk 0.865 @ 4.7; F-NG 0.728 @ 14.5; F-G 0.884 @ 5.3; J-NG 0.759 @ 13.9; J-G 0.884 @ 5.3.

Random left-regular, competitive ratio at selected d**:**

d	SG	Rk	F-NG	F-G	J-NG	J-G
1	1.000	1.000	0.985	1.000	0.985	1.000
2	0.954	0.954	0.877	0.968	0.894	0.966
5	**0.890**	0.890	0.758	0.900	0.788	0.901
10	0.927	0.928	0.733	0.928	0.766	0.928
30	0.977	0.977	0.730	0.976	0.760	0.976

Paper-claim checklist (all verified ✓): greedy minima near $c \approx 4.9$ / $d = 5$; Ranking $\approx$ SimpleGreedy (max diff 0.0017 ER, 0.0013 LR; the paper omits Ranking's curve for this reason);

non-greedy variants degrade monotonically as c, d grow; greedy complex variants $\approx$ SimpleGreedy asymptotically; the c=14.9 non-greedy ordering (J-NG 0.760 > F-NG 0.730) matches the paper's worst-case-bound ordering. Across families, the non-greedy algorithms converge to the same asymptotic constants (0.730 Feldman, 0.760 Jaillet–Lu, within 0.001), above their worst-case bounds by +0.06 / +0.03; §3.6 reads that observation.

5 Data provenance

Real data is stored locally under `data/` (large; excluded from version control).

- **Real graphs (Chapter 7, §3.6 validation):** six graphs from the Network Repository (`networkrepository.com`): `socfb-Caltech36`, `socfb-Reed98`, `bio-CE-GN`, `bio-CE-PG`, `econ-beause`, `econ-mbeaflw`, downloaded as MatrixMarket `.mtx` / whitespace `.edges`, reduced to simple undirected graphs and converted to bipartite by random balanced partition (Borodin Table 3) or duplicating double-cover (Table 4).
- **Traces (Chapters 7, 8, 9):** Wikipedia "top articles per day" for four days, retrieved from the Wikimedia REST pageviews API (`data/trace/wiki/`, used as live day vs 1/7/30-day-stale forecasts); the Azure LLM inference trace from Microsoft's public Azure dataset release (`data/trace/azure_llm/`, context/generated token counts with timestamps); and the Mooncake conversation trace released with [18] (`data/trace/mooncake/`, per-request `hash_ids` for prefix-cache blocks). The Wikipedia trace is a snapshot of a live source rather than a static benchmark, so exact reproduction needs the same snapshot, not merely the same API.

6 Outlook verification snippets (§10.2)

Every quantity quoted in the outlook of §10.2 was checked numerically before being used. There are three checks, each a short simulation of the rare-resource construction.

1. **Per-cell constants.** The per-cell optimum, the per-cell baseline, and the closed forms for the advantage and for ℓ_1 agree with simulation to three decimals. For example, at contention

$\theta = 0.6$ and advice bias 0.3 the simulated per-cell advantage is ± 0.119, against a predicted $\pm\theta \cdot 0.3 = \pm 0.12$.

2. **The conversion law.** The expected ratio when the advice is followed matches $\rho_{\text{perfect}} - \frac{1}{2}\ell_1(p, q)$, where ρ_{perfect} is the ratio under perfect advice, i.e. it falls off linearly in the advice's ℓ_1 error, again to three decimals. In the language of §10.2 this is what makes the *stakes* δ a linear function of the advice error.
3. **The two budget–stakes ingredients.** The directional statistic's accuracy at a fixed prefix length does not degrade as n grows, and the plug-in ℓ_1 estimate becomes blind, i.e. indistinguishable between good and bad advice, once $k \ll r$. Both are produced by `scripts/verify_witness_gap.py` (§A.2).

These checks are what license the *reading* of §10.2; they are not a proof of it. The formal development is separate work outside this thesis.

7 The relaxation ladder: streaming and offline algorithms

The original project brief suggested extending the reproduction with *streaming or offline algorithms*. My proposal instead made the learning-augmented family the main extension. This appendix supplies the suggested arm at reduced scope and connects it to the thesis's central question: is the small remaining gap to the optimum caused by missing information, or by the irrevocability of online decisions?

The comparison is a **ladder of relaxations** on the same instances. Online vertex arrival reveals one node's neighbourhood at a time and requires an immediate, irrevocable match. The semi-streaming variant instead receives edges in a uniformly random order and stores only $O(n)$ words. Its first pass is greedy; each additional pass applies vertex-disjoint length-3 augmentations [7, 15]. The offline rung is the Hopcroft–Karp optimum already used as the denominator throughout the thesis. This is not a claim that streaming is an online algorithm: extra passes explicitly relax irrevocability.

Setup. The ladder uses the three synthetic families of Table 4.1 and the six real graphs of §7.2, with the same instances, optimum and 95% interval convention. Its online rows reproduce

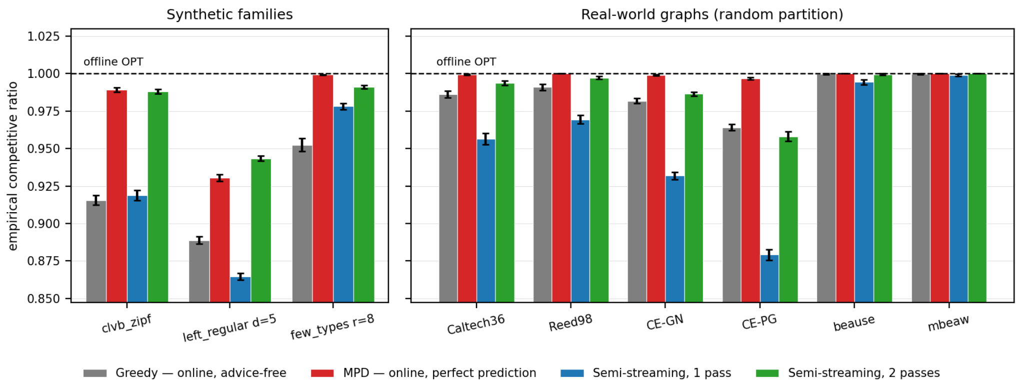


Figure 1: The relaxation ladder. Grey and red are the online model (advice-free Greedy, and MPD given a perfect degree prediction); blue and green are semi-streaming at one and two passes. On seven of the nine graphs one extra pass buys more than a perfect prediction does.

Table 4.1 within the reported intervals. Full numerical outputs are in `results/streaming_ladder_tables.md`; Figure A.1 shows the four rungs needed for the comparison.

The important comparison is marginal. Perfect degree advice improves the online baseline; one revision pass improves the one-pass streaming baseline. One revision is worth more than perfect advice on seven of the nine graphs, including all six real ones. It lifts every graph to within 0.6–4.2% of the optimum. The exceptions are `clvb_zipf`, where the gains differ by less than 0.005, and `few_types`, the synthetic family deliberately constructed to reward degree information. Thus, on these average-case inputs, the residual gap is more responsive to revision than to better prediction. This supports the thesis's interpretation that prediction has little performance headroom because the advice-free baseline is already strong; the remaining loss is largely the price of irrevocability rather than an information deficit.

The comparison is deliberately limited. Because dispatched decisions cannot be recalled, the second pass is unavailable in the motivating applications; the experiment compares the *size* of the two gains, not their practical interchangeability. It tests one simple family under random rather than adversarial edge order, without stronger $(1-\varepsilon)$-approximation streaming algorithms. It is evidence about where the gap lies, not a full benchmark.